ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP LỚN

KỸ THUẬT DỮ LIỆU (CO5173)

Data Engineering

Đề tài: Project

NỀN TẢNG KỸ THUẬT DỮ LIỆU CHO HỆ THỐNG GIÁO DỤC TRỰC TUYẾN QUY MÔ LỚN

Learning Analytics, gợi ý học tập và cảnh báo sớm nguy cơ bỏ học

---

Giảng viên hướng dẫn	PGS. TS. Võ Thị Ngọc Châu
Lớp / Nhóm	[Lớp] / [Nhóm]
Học kỳ	Học kỳ 1, năm học 2026–2027

Học viên thực hiện

Họ và tên	**MSHV**	**Phần việc chính**
[Phạm Duy Anh]	[2310139]	[BR4, BR5]
[Họ và tên thành viên 2]	[MSHV]	[Phần việc]
[Họ và tên thành viên 3]	[MSHV]	[Phần việc]
[Họ và tên thành viên 4]	[MSHV]	[Phần việc]

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

Tóm tắt **9**

Danh mục thuật ngữ và từ viết tắt **10**

1 Bài toán, stakeholders và yêu cầu **12**

1.1 Giới thiệu 12

1.1.1 Bối cảnh 12

1.1.2 Phát biểu bài toán 12

1.1.3 Mục tiêu và phạm vi 13

1.1.4 Đóng góp chính 13

1.1.5 Phương pháp thực hiện 14

1.1.6 Cấu trúc báo cáo 14

1.2 Phân tích yêu cầu và nguồn dữ liệu 14

1.2.1 Các bên liên quan và yêu cầu nghiệp vụ 14

1.2.2 Nguồn dữ liệu 15

1.2.3 Yêu cầu phi chức năng 16

1.2.4 Ánh xạ yêu cầu vào pipeline 17

2 Kiến trúc và lựa chọn công nghệ **19**

2.1 Kiến trúc giải pháp 19

2.1.1 Tổng quan kiến trúc 19

2.1.2 Lựa chọn công nghệ 20

2.1.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác 22

2.1.4 Ngữ nghĩa xử lý luồng 22

2.1.5 Các quyết định kiến trúc 23
2.1.6 Điều chỉnh so với kiến trúc tham chiếu 24

3 Lakehouse và kho dữ liệu 26
3.1 Tổng quan lakehouse 26
3.1.1 Kiến trúc medallion 26
3.1.2 Chuẩn hoá telemetry và tiến hoá schema 27
3.1.3 Khử định danh và phân quyền 28
3.1.4 Mô hình chiều Kimball ở lớp gold 28
3.1.5 Data mart theo yêu cầu nghiệp vụ 30
3.1.6 Thiết kế vật lý 31
3.1.7 Cổng kiểm tra chất lượng dữ liệu 31
3.1.8 Điều phối, dữ liệu trễ và chạy lại 32

4 Hiện thực BR1–BR8 và ứng dụng 35
4.1 Thiết kế và hiện thực theo từng yêu cầu 35
4.1.1 BR1 — Pipeline thông báo hướng sự kiện 35
4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo 36
4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết 37
4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường 38
4.1.5 BR5 — Cảnh báo sớm nguy cơ bỏ học 40
4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức 41
4.1.7 BR7 — Heatmap tương tác video 41
4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác 42
4.1.8.1 Đồng bộ cache và chỉ mục tìm kiếm 42
4.1.8.2 Xu hướng tương tác 42
4.2 Giao diện người dùng 43
4.2.1 Quy trình thiết kế 43
4.2.2 Đáp ứng yêu cầu về giao diện 44
4.2.3 Hệ thống thiết kế 46
4.2.4 Kiến trúc frontend 46

4.2.5 Các màn hình chính . . . 47
4.2.6 Kiểm thử giao diện . . . 52

5 Kiểm thử và đánh giá 54
5.1 Chiến lược kiểm thử . . . 54
5.2 Kiểm chứng nghiệp vụ . . . 55
5.3 Môi trường đo . . . 57
5.4 Kết quả hiệu năng . . . 59
5.4.1 BR1–BR3: độ trễ đầu cuối . . . 59
5.4.2 BR4: tổng hợp SQL so với Python . . . 60
5.4.3 BR5: độ trễ đầu cuối và thông lượng cửa sổ . . . 61
5.4.4 BR6–BR8: phục vụ mart . . . 62
5.4.5 BR8: từ xuất bản đến khi tìm kiếm được . . . 62
5.4.6 BR7/BR8: lớp phủ thời gian thực . . . 63
5.4.7 Thời gian build kho dữ liệu . . . 63
5.5 Debezium và polling: negative-control benchmark . . . 64
5.5.1 Thiết lập . . . 64
5.5.2 Kết quả . . . 65
5.5.3 Ý nghĩa với Big Data . . . 65
5.5.4 Giới hạn phép đo . . . 66
5.6 Thảo luận . . . 66

6 Hạn chế, hướng phát triển và kết luận 68
6.1 Hạn chế hiện tại . . . 68
6.2 Hướng phát triển . . . 69
6.3 Kết luận . . . 69

Tài liệu tham khảo 71

A Hướng dẫn chạy và tái lập kết quả 73
A.1 Yêu cầu . . . 73
A.2 Khởi động . . . 73

A.3 Kiểm thử 75
A.4 Đo hiệu năng 75
A.5 Đo Debezium và polling 76
A.6 Biên dịch báo cáo 77

B Tóm tắt API **78**

Danh sách hình vẽ

2.1 Kiến trúc tổng thể của nền tảng Project 20

3.1 Các lớp của lakehouse và tần suất cập nhật 26

4.1 Trích tập luật thông báo 36
4.2 Đường CDC thật của BR4 và BR5 39
4.3 Các cửa sổ đặc trưng tại mốc đánh giá T (BR5) 40
4.4 Đồng bộ nội dung BR8 dựa trên CDC 42
4.5 Mockup: Dashboard người học với thẻ khoá học (BR2), thông báo (BR1) và phản hồi gần đây (BR3) 44
4.6 Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff 48
4.7 Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20% 48
4.8 Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video “Joins” (390 giây), kèm các thời điểm cần xem lại 49
4.9 Learners to support (BR5): hàng đợi rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15 49
4.10 Organisation progress (BR6): k-anonymity với $k = 5$ 50
4.11 Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được 51
4.12 Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu . . . 51
4.13 Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang 52

5.1 BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log) . . 61

5.2 Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp) 62
5.3 BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp 62
5.4 Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần) . . 63

Danh sách bảng

1.1 Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu 14
1.2 Bốn nguồn dữ liệu và cách tiếp nhận 16
1.3 Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime 17

2.1 Công nghệ theo vai trò 21
2.2 Các lớp trong một bounded context 22
2.3 Tóm tắt các quyết định kiến trúc (ADR) 23

3.1 Hợp đồng dữ liệu của từng lớp 27
3.2 Phân quyền truy cập kho dữ liệu qua Trino 28
3.3 Ma trận bus của lớp gold (D: vai trò ngày, •: khoá chiều) 29
3.4 Các data mart và kho đặc trưng 30
3.5 Các cổng kiểm tra chất lượng 32
3.6 Các DAG Airflow 33
3.7 Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự 33

4.1 Các tầng của luật gợi ý (BR2) 37
4.2 Quy tắc phát hiện bất thường của BR4 (cấu hình được) 39
4.3 Các tín hiệu rủi ro và điểm (BR5) 40
4.4 Trọng số hoạt động trong điểm tương tác (`EngagementWeights`) 43
4.5 Đặc điểm người dùng và lựa chọn thiết kế tương ứng 45
4.6 Workspace, đường dẫn và trang 46

5.1 Các tầng kiểm thử 55
5.2 Các kịch bản kiểm chứng nghiệp vụ và kết quả 56

5.3 Container chạy trong từng phép đo (stack đầy đủ: 30 container) 58
5.4 Môi trường đo hiệu năng 59
5.5 Độ trễ BR1–BR3 (ms) 60
5.6 BR5: chặng Kafka → Spark → cửa sổ đã lưu (trung bình 3 lần) 61
5.7 Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước) 63
5.8 Debezium và polling trên cùng 100.000 dòng 65

B.1 Các endpoint chính 78

Tóm tắt

Báo cáo trình bày thiết kế và hiện thực **Project**, nền tảng kỹ thuật dữ liệu cho hệ thống giáo dục trực tuyến quy mô lớn. Nền tảng tiếp nhận bốn nguồn dữ liệu rời rạc — telemetry clickstream từ web và mobile, CDC từ PostgreSQL, kho học liệu phi cấu trúc và nhật ký sandbox — rồi chuyển hoá thành kho dữ liệu phục vụ tám yêu cầu nghiệp vụ (BR1–BR8) của bốn nhóm người dùng: người học, giảng viên, cố vấn học tập và quản trị viên tổ chức.

Kiến trúc dùng Kafka và Debezium làm tầng tiếp nhận và event bus; Spark (batch và Structured Streaming) là engine xử lý chính; Delta Lake trên MinIO tổ chức theo mô hình medallion bronze–silver–gold, với lớp gold là kho dữ liệu hình sao Kimball; Hive Metastore lập danh mục và Trino phục vụ SQL; Redis đảm nhận phục vụ độ trễ thấp, OpenSearch tìm kiếm lai. Mã nguồn tổ chức thành bảy *bounded context* theo DDD và kiến trúc lục giác, cho phép cùng một use case chạy trên nhiều engine và kiểm thử độc lập với engine. Mười sáu ADR ghi lại các quyết định, gồm những điều chỉnh so với kiến trúc tham chiếu. BR1–BR3 chạy end-to-end trên Kafka, Redis, PostgreSQL và Delta; BR4–BR8 chạy trên hạ tầng lakehouse.

Về kết quả đo trên máy cá nhân với dữ liệu tổng hợp:

- **BR1–BR3**: trung vị 12,4 ms từ sự kiện đến thông báo, 2,2 ms cho API gợi ý, 14,2 ms để nhận 202 và 71,0 ms từ nộp bài đến khi chấm xong.
- **BR5**: từ thay đổi CDC đến cảnh báo bỏ học mất khoảng 10,2 giây (kịch bản chấp nhận, không đo lại).
- **BR8**: nội dung xuất bản được lập chỉ mục sau trung vị 501 ms.
- **BR7–BR8**: lớp phủ hiển thị sau khoảng 6 giây, ổn định khi tải tăng 15 lần.
- **BR6–BR8**: mart qua Trino 81–172 ms; qua API có cache 2,6–6,0 ms.
- **Kho dữ liệu**: 43 model xây trong khoảng 2 phút (121–130 giây), đạt mọi cổng chất lượng.

Hệ thống có 379 test đơn vị, 1 test tích hợp end-to-end, 12 test trình duyệt cùng các kịch bản chấp nhận; báo cáo cũng trình bày giao diện web, các giới hạn còn lại và hướng phát triển.

Từ khoá: kỹ thuật dữ liệu, lakehouse, Delta Lake, Kafka, Debezium, CDC, Spark Structured Streaming, learning analytics.

Danh mục thuật ngữ và từ viết tắt

Thuật ngữ	Giải thích
ADR	*Architecture Decision Record*: tài liệu ghi lại một quyết định kiến trúc, bối cảnh, phương án bị loại và hệ quả.
BR	*Business Requirement*: yêu cầu nghiệp vụ (BR1–BR8) do nhóm sinh viên lựa chọn.
Bronze / Silver / Gold	Ba lớp của medallion: dữ liệu thô, dữ liệu sạch và khử định danh, dữ liệu mô hình hoá phục vụ phân tích.
CDC	*Change Data Capture*: bắt các thay đổi cấp dòng từ nhật ký giao dịch (WAL) của cơ sở dữ liệu.
CDF	*Change Data Feed*: tính năng của Delta Lake ghi lại các thay đổi cấp dòng của bảng.
DDD	*Domain-Driven Design*: thiết kế hướng miền nghiệp vụ, chia hệ thống thành các *bounded context*.
DLQ	*Dead-Letter Queue*: nơi lưu sự kiện bị từ chối kèm lý do để xử lý lại.
Event time	Thời điểm sự kiện xảy ra tại nguồn, khác với thời điểm hệ thống nhận được.
Exactly-once	Đảm bảo mỗi bản ghi tác động lên kết quả đúng một lần dù có lỗi hoặc gửi lại.
Hexagonal	Kiến trúc lục giác (ports and adapters): lõi nghiệp vụ chỉ phụ thuộc vào các cổng trừu tượng.
HMS	Hive Metastore: dịch vụ danh mục (catalog) chung cho các bảng Delta.
Idempotent	Thực thi lặp lại nhiều lần cho cùng kết quả như thực thi một lần.
k-anonymity	Nguyên tắc không công bố số liệu của nhóm có ít hơn k cá thể; ở đây $k = 5$.

Thuật ngữ	Giải thích
LSN	*Log Sequence Number*: vị trí trong WAL của PostgreSQL, dùng để sắp thứ tự thay đổi.
Mart	Bảng tổng hợp phục vụ một yêu cầu nghiệp vụ cụ thể.
Medallion	Mô hình tổ chức lakehouse theo các lớp chất lượng tăng dần.
OLTP	*Online Transaction Processing*: cơ sở dữ liệu giao dịch của hệ thống quản lý học tập (LMS).
PII	*Personally Identifiable Information*: thông tin định danh cá nhân.
Pseudonym	Bút danh: giá trị băm có khoá (HMAC) thay cho định danh thật.
SCD	*Slowly Changing Dimension*: chiến lược lưu lịch sử thay đổi của bảng chiều (loại 1 ghi đè, loại 2 lưu phiên bản).
Watermark	Ngưỡng thời gian sự kiện mà engine streaming coi là đã nhận đủ dữ liệu trước đó.

Chương 1

Bài toán, stakeholders và yêu cầu

1.1 Giới thiệu

1.1.1 Bối cảnh

Nền tảng giáo dục trực tuyến quy mô lớn sinh ra dữ liệu liên tục và đa dạng. Mỗi thao tác của người học — mở bài giảng, tua video, nộp bài lập trình, thảo luận — đều để lại dấu vết. LMS ghi nhận tiến độ, bài nộp và điểm số; kho học liệu chứa transcript, slide và PDF được cập nhật theo phiên bản; sandbox chấm bài sinh ra nhật ký thực thi chi tiết. Dữ liệu này có giá trị lớn cho *learning analytics*: giúp giảng viên phát hiện bài tập có vấn đề, cố vấn nhận diện sớm nguy cơ bỏ học và người học nhận phản hồi kịp thời.

Tuy nhiên, các nguồn dữ liệu rời rạc và khác nhau về định dạng, tốc độ, độ tin cậy. Telemetry từ thiết bị di động có thể đến trễ nhiều ngày hoặc sai thứ tự; thay đổi trong cơ sở dữ liệu giao dịch phải được bắt đúng thứ tự mà không làm nặng hệ thống production; tài liệu học tập cần chia đoạn và quản lý phiên bản; dữ liệu cá nhân phải được bảo vệ, nhất là trong báo cáo cho tổ chức đối tác (B2B). Đây là bài toán kỹ thuật dữ liệu điển hình, đòi hỏi kết hợp xử lý luồng, xử lý lô, lưu trữ phân tích và phục vụ độ trễ thấp [1].

1.1.2 Phát biểu bài toán

Project hướng tới xây dựng nền tảng dữ liệu biến bốn nguồn thô thành kho dữ liệu phục vụ ba mục tiêu: phân tích học tập, gợi ý thích ứng và cảnh báo sớm nguy cơ bỏ học. Từ yêu cầu thiết kế một hệ thống dữ liệu trong giáo dục, nhóm sinh viên lựa chọn tám yêu cầu kỹ thuật dữ liệu (BR1–BR8) cho bốn nhóm người dùng và chọn kiến trúc tham chiếu: Kafka và Debezium cho tiếp nhận, Flink cho xử lý luồng, Spark cho xử lý lô, Delta Lake cho lakehouse, Redis cho tầng

phục vụ. Các mục sau phân tích chi tiết các yêu cầu này.

1.1.3 Mục tiêu và phạm vi

Mục tiêu của nhóm gồm:

1. Thiết kế kiến trúc dữ liệu đầu cuối bao phủ tám yêu cầu nghiệp vụ, bám sát kiến trúc tham chiếu; mọi điều chỉnh đều có lý do và được ghi lại.
2. Hiện thực pipeline chạy trên máy cá nhân bằng Docker Compose: CDC thật qua Debezium, lakehouse Delta Lake với kho dữ liệu hình sao, data mart theo từng BR và các đường xử lý thời gian thực.
3. Bảo đảm các tính chất cốt lõi: xử lý theo thời gian sự kiện, chịu dữ liệu trễ và trùng lặp, chạy lại idempotent, khử định danh và k-anonymity, kiểm tra chất lượng dữ liệu.
4. Cung cấp API và giao diện web cho từng nhóm người dùng để kết quả pipeline được sử dụng thực tế.
5. Đánh giá tính đúng đắn và hiệu năng bằng kiểm thử tự động và đo đạc tái lập được.

Phạm vi: báo cáo dùng **dữ liệu tổng hợp** (synthetic); hệ thống chạy trên một máy, không nhằm chứng minh mở rộng ngang hay độ sẵn sàng cao. Các số đo hiệu năng là quan sát cục bộ, không phải cam kết năng lực.

1.1.4 Đóng góp chính

- **Lakehouse Delta Lake** ba lớp với 51 bảng và 44 model (43 model tại thời điểm đo thời gian build ở Chương 5, trước khi bổ sung model lịch sử chuyển trạng thái cảnh báo của BR5); lớp gold là kho dữ liệu Kimball gồm 9 bảng chiều và 11 bảng sự kiện, dùng khoá thay thế tất định; các mart cho BR4, BR6, BR7, BR8 và kho đặc trưng ngoại tuyến cho BR2; Hive Metastore lập danh mục, Trino phục vụ truy vấn có phân quyền theo vai trò.
- **Đường chạy thật cho BR1–BR3**: dịch vụ thông báo và chấm bài trên Kafka, Redis, PostgreSQL; feature store hai nhánh cho gợi ý.
- Đường **CDC thật** từ PostgreSQL WAL qua Debezium, Kafka và Spark Structured Streaming đến mart, trạng thái rủi ro và cảnh báo cho BR4–BR5; đường này chịu được replay, cập nhật, xoá và khởi động lại checkpoint.
- Đồng bộ nội dung **BR8 dựa trên CDC**: vô hiệu hoá cache Redis, chia đoạn, nhúng vector vào OpenSearch và tìm kiếm lai, idempotent theo phiên bản.
- **Lớp phủ thời gian thực** cho heatmap video (BR7) và điểm tương tác (BR8) trên Redis, ghép với mart theo ngày.

- Mười sáu **ADR**, kiến trúc DDD/hexagonal có test bảo vệ ranh giới và bộ kiểm thử nhiều tầng kèm bằng chứng đo đạc.
- **Giao diện web** cho năm workspace, thiết kế từ mockup và nối với API thật.

1.1.5 Phương pháp thực hiện

Nhóm làm việc theo vòng lặp ngắn: phân tích yêu cầu, ghi ADR, hiện thực, kiểm chứng bằng test và số đo. Lõi nghiệp vụ (*policy* và *use case*) được viết trước và kiểm thử độc lập với engine; sau đó bổ sung dần adapter cho Kafka, Spark, Delta, Redis, PostgreSQL và OpenSearch. Mỗi nhóm BR có tài liệu kế hoạch và tài liệu hiện thực riêng, ghi rõ phần đã hiện thực, phần còn hạn chế và lỗi phát hiện khi kiểm chứng.

1.1.6 Cấu trúc báo cáo

Báo cáo gồm sáu chương:

- Chương 1: người dùng, yêu cầu nghiệp vụ, nguồn dữ liệu và yêu cầu phi chức năng.
- Chương 2: kiến trúc, lựa chọn công nghệ và các quyết định thiết kế.
- Chương 3: lakehouse, mô hình medallion, kho dữ liệu và data mart.
- Chương 4: hiện thực BR1–BR8 và giao diện ứng dụng.
- Chương 5: kiểm thử, đánh giá hiệu năng và benchmark Debezium–polling.
- Chương 6: hạn chế, hướng phát triển và kết luận.

1.2 Phân tích yêu cầu và nguồn dữ liệu

1.2.1 Các bên liên quan và yêu cầu nghiệp vụ

Nhóm xác định bốn nhóm người dùng và tám yêu cầu nghiệp vụ. Bảng 1.1 tóm tắt từng yêu cầu và nhu cầu kỹ thuật tương ứng theo cách đọc của nhóm.

Bảng 1.1. Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu kỹ thuật dữ liệu
BR1	Người học	Nhận thông báo khi có bài tập mới, deadline sắp đến hoặc nội dung được cập nhật	Pipeline thông báo hướng sự kiện: nhận sự kiện LMS qua message broker, lọc bằng rule engine, gửi theo thời gian thực

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu kỹ thuật dữ liệu
BR2	Người học	Nhận gợi ý khoá học và bước học tiếp theo	Pipeline đặc trưng hai nhánh: đặc trưng theo lô (Spark) và đặc trưng phiên (streaming) qua feature store
BR3	Người học	Nhận phản hồi chi tiết khi nộp bài lập trình/SQL	Tiếp nhận bài nộp tách rời qua hàng đợi đệm; chuẩn hoá kết quả kiểm thử (stderr, diff test case, thời gian chạy)
BR4	Giảng viên	Phát hiện câu hỏi hoặc test case có tỷ lệ trượt bất thường	Aggregation mart: tỷ lệ đạt theo từng test case và câu hỏi để lọc đề lỗi logic hoặc quá khó
BR5	Cố vấn học tập	Nhận cảnh báo sớm học viên có nguy cơ bỏ dở	CDC và cửa sổ trượt: đọc log thay đổi từ OLTP, tính điểm rủi ro, phát sinh cảnh báo
BR6	Quản trị viên (B2B)	Theo dõi tiến độ kỹ năng và tỷ lệ hoàn thành theo tổ chức	Pipeline medallion: chuẩn hoá telemetry có phiên bản schema, khử định danh, báo cáo tổ chức với ngưỡng nhóm tối thiểu
BR7	Giảng viên	Xem heatmap tương tác video bài giảng	Tổng hợp chi tiết: mật độ play/pause/tua theo từng giây video
BR8	Giảng viên	Đồng bộ trạng thái, vô hiệu hoá cache, làm mới chỉ mục tìm kiếm khi nội dung thay đổi	Pipeline xu hướng tương tác: tổng hợp hoạt động theo cửa sổ thời gian, tính điểm tương tác, phát hiện xu hướng giảm

Nhận xét về BR8

Yêu cầu nghiệp vụ của BR8 (đồng bộ cache và chỉ mục khi nội dung thay đổi) và nhu cầu kỹ thuật (pipeline xu hướng tương tác) là hai nhu cầu khác nhau. Nhóm hiện thực **cả hai**: đồng bộ nội dung dựa trên CDC (Mục 4.1.8.1) và mart xu hướng tương tác (Mục 4.1.8.2).

1.2.2 Nguồn dữ liệu

Bảng 1.2 mô tả bốn nguồn dữ liệu thô, đặc tính và cách nền tảng tiếp nhận. Mỗi nguồn có thách thức riêng, quyết định thiết kế tầng tiếp nhận.

Bảng 1.2. Bốn nguồn dữ liệu và cách tiếp nhận

Nguồn	Đặc tính	Thách thức	Tiếp nhận
Telemetry (web, Android, iOS)	JSON bán cấu trúc kiểu xAPI [2], lưu lượng lớn	Đến trễ, sai thứ tự (thiết bị offline), gửi lại, nhiều phiên bản schema, chứa PII, đồng hồ thiết bị lệch	Topic `telemetry.events.raw.v1`, khoá theo người học; DLQ `telemetry.events.dlq.v1`
CDC từ PostgreSQL	Thay đổi cấp dòng từ WAL: tiến độ, bài nộp, điểm	Giữ thứ tự theo khoá, không ảnh hưởng truy vấn production, xử lý cập nhật và xoá	Debezium (plugin `pgoutput`, replication slot) vào 11 topic `cdc.lms.public.*`
Kho học liệu (object storage)	Transcript, slide, PDF; văn bản dài	Chia đoạn theo ngữ nghĩa, quản lý phiên bản nghiêm ngặt	Đối tượng bất biến theo phiên bản trong MinIO; thay đổi đi qua CDC của bảng `course_content`
Nhật ký sandbox	stdout, stderr, thời gian chạy, bộ nhớ, trạng thái assertion	Chuẩn hoá đầu ra, tách lỗi hạ tầng khỏi lỗi của người học	Topic `grading.submission_received.v1`, `grading.sandbox_result.raw.v1`, `grading.submission_graded.v1`

Danh mục Kafka gồm 26 topic, khai báo dưới dạng mã. Ngoài các topic trên còn có topic sự kiện miền: `lms.*` (bài tập mới, sắp đến hạn, khoá học cập nhật), `risk.*` (điểm rủi ro, cảnh báo), `notification.*` và `content.*`. Runtime BR4/BR5 tự tạo thêm các topic của riêng nó khi khởi tạo, ngoài danh mục này: chín topic CDC `br45.cdc.br45.<table>`, `br45.activity.v1`, `br45.clean.v2`, `br45.analytics.v2` và `risk.alert_transition.v1`.

1.2.3 Yêu cầu phi chức năng

Từ đặc tính nguồn dữ liệu và nhu cầu người dùng, nhóm rút ra các yêu cầu phi chức năng sau, dùng làm tiêu chí đánh giá các quyết định kiến trúc ở Chương 2.

Độ trễ phù hợp từng BR

Thông báo, phản hồi chấm bài và cảnh báo rủi ro cần độ trễ tính bằng giây. Báo cáo tổ chức

và heatmap chấp nhận cập nhật theo ngày, kèm lớp phủ cho hôm nay. Tra cứu đặc trưng cho gợi ý cần dưới 10 ms.

Đúng một lần, idempotent

Gửi lại từ Kafka, chạy lại job và backfill không được làm sai số liệu; mọi lệnh batch phải idempotent theo ngày chạy.

Xử lý theo thời gian sự kiện

Cửa sổ thời gian tính theo thời điểm sự kiện, có watermark; dữ liệu trễ vẫn được lưu và gắn cờ, không bị bỏ qua.

Quyền riêng tư

Định danh thật chỉ tồn tại ở lớp thô và bị hạn chế truy cập; lớp phân tích chỉ dùng bút danh; báo cáo tổ chức không công bố số liệu của nhóm dưới $k = 5$ người.

Tiến hoá schema

Payload client có nhiều phiên bản; hệ thống chuyển đổi về phiên bản hiện hành hoặc từ chối kèm lý do.

Tái lập và kiểm toán

Có thể dựng lại dữ liệu từ lớp thô, xem trạng thái cũ (time travel) và truy vết mỗi lần build.

Chất lượng dữ liệu

Cổng kiểm tra sau mỗi lớp; lỗi nghiêm trọng dừng pipeline.

Chạy được trên máy cá nhân

Toàn hệ thống chạy bằng Docker Compose, chỉ dùng CPU và không phụ thuộc dịch vụ trả phí.

1.2.4 Ánh xạ yêu cầu vào pipeline

Bảng 1.3 ánh xạ mỗi BR vào bounded context, runtime xử lý và nguồn phục vụ; cột cuối ghi mức hoàn thiện thực tế, được giải thích ở Chương 6.

Bảng 1.3. Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime

BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
1	notification	Dịch vụ Kafka + rule engine; fan-out theo PostgreSQL	Redis (dedup, inbox); SSE	Chạy thật
2	recommendation	Spark batch + session streaming; xếp hạng bằng luật	Delta, Redis, API	Chạy thật

BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
3	grading	Kafka + grading-ingest/worker/normalizer	Redis; PostgreSQL `grading.*`	Chạy thật
4	analytics + br45	CDC, bảng Delta, job Spark SQL bất đồng bộ	PostgreSQL `analytics`; lịch sử ở `mart.assessment_item_stats`	Chạy thật
5	risk + br45	CDC, Spark Structured Streaming (cửa sổ 7 ngày)	PostgreSQL; lịch sử ở `gold.fct_risk_score_daily`	Chạy thật
6	telemetry + analytics	Medallion Spark	Delta, Trino (`mart.org_progress`)	Chạy thật
7	analytics	Spark batch + lớp phủ streaming	Delta, Trino, Redis	Chạy thật
8	content + analytics	CDC, dịch vụ content-sync; Spark batch + lớp phủ	Redis, OpenSearch, Delta, Trino	Chạy thật

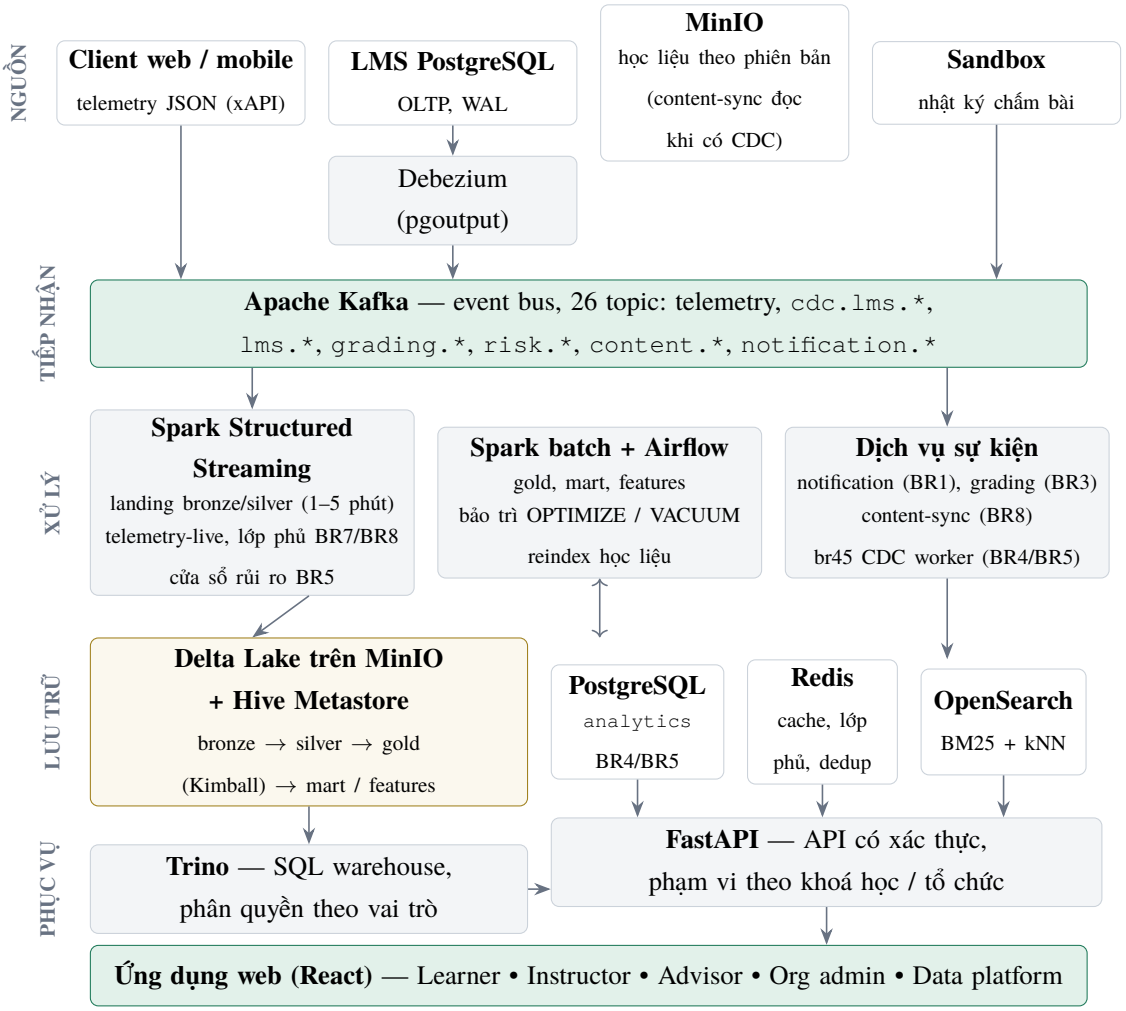
Chương 2

Kiến trúc và lựa chọn công nghệ

2.1 Kiến trúc giải pháp

2.1.1 Tổng quan kiến trúc

Hình 2.1 mô tả kiến trúc đầu cuối gồm sáu tầng. Tầng nguồn gồm bốn nguồn dữ liệu; tầng tiếp nhận dùng Kafka làm event bus duy nhất, với Debezium bắt thay đổi từ WAL của PostgreSQL; tầng xử lý dùng Spark cho cả luồng và lô cùng các dịch vụ tiêu thụ sự kiện; tầng lưu trữ gồm Delta Lake, PostgreSQL cho mart thời gian thực BR4–BR5, Redis và OpenSearch; tầng phục vụ gồm Trino và API FastAPI; trên cùng là ứng dụng web cho năm nhóm người dùng.



Hình 2.1. Kiến trúc tổng thể của nền tảng Project

Ba nguyên tắc xuyên suốt kiến trúc:

- **Kafka là ranh giới giữa các context.** Các bounded context chỉ giao tiếp qua topic Kafka và bảng lakehouse, không dùng chung cơ sở dữ liệu. Nhờ đó mỗi context có thể triển khai và thay đổi độc lập.
- **Spark là bên ghi duy nhất của lakehouse** (ADR 0011). Các job thời gian thực chỉ ghi ra Kafka và Redis. Điều này tránh xung đột commit khi nhiều engine ghi cùng bảng Delta và giữ logic idempotent ở một nơi.
- **Lakehouse là nguồn sự thật cho phân tích.** Redis và các mart phục vụ đều dựng lại được từ lakehouse hoặc từ dữ liệu thô (ADR 0006).

2.1.2 Lựa chọn công nghệ

Bảng 2.1 liệt kê công nghệ theo vai trò. Thành phần chính gồm Kafka [3], Debezium [4], Spark [5] với Structured Streaming [6], Delta Lake [7], Trino [8], Redis [9] và OpenSearch [10].

Nhóm giữ các lựa chọn công nghệ ban đầu nếu chạy được trên máy cá nhân; khi thay thế, chọn thành phần cùng vai trò và ghi lại bằng ADR.

Bảng 2.1. Công nghệ theo vai trò

Vai trò	Công nghệ	Lý do chọn
Tiếp nhận, event bus	Apache Kafka	Bộ đệm cho telemetry tốc độ cao, giữ thứ tự theo khoá, phát lại được; dùng chung làm event bus cho các sự kiện LMS (ADR 0003)
CDC	Debezium (pgoutput)	Đọc replication slot nên không tạo tải truy vấn lên production; giữ LSN để sắp thứ tự
Xử lý luồng	Spark Structured Streaming	Watermark theo event time, checkpoint, sink giao dịch Delta; dùng chung engine với batch (ADR 0009, 0014)
Xử lý lô	Apache Spark 3.5	Hệ sinh thái Delta, xử lý song song cho model SQL và chia đoạn học liệu (ADR 0004)
Lakehouse	Delta Lake 3.3 trên MinIO	ACID MERGE, time travel, Change Data Feed (ADR 0005)
Catalog	Hive Metastore 3.1.3	Danh mục chung cho Spark và Trino; bản 3.1 tương thích Spark 3.5 (ADR 0010)
SQL warehouse	Trino	Truy vấn trực tiếp bảng Delta, ANSI SQL, phân quyền theo schema, không cần bản sao thứ hai (ADR 0010)
Phục vụ độ trễ thấp	Redis	Tra cứu theo khoá dưới mili-giây, TTL; dùng cho cache, lớp phủ thời gian thực, dedup thông báo (ADR 0006)
Mart thời gian thực BR4/BR5	PostgreSQL 16	Phục vụ giao dịch; sau mỗi thay đổi của nguồn, kết quả được cập nhật bất đồng bộ, mỗi lần trong một giao dịch (ADR 0009)
Tìm kiếm	OpenSearch 2.17	BM25 kết hợp vector HNSW; xếp hạng lai bằng reciprocal rank fusion (ADR 0013)
Điều phối	Apache Airflow	DAG theo lịch và theo dataset; mọi task idempotent theo ngày chạy
API, giao diện	FastAPI, React 18 + TypeScript	Một API chung cho mọi BR; giao diện không dùng thư viện UI hay biểu đồ ngoài

2.1.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác

Mã nguồn nằm trong một gói Python duy nhất (monorepo, ADR 0002), chia thành bảy bounded context theo Domain-Driven Design [11]: `telemetry`, `notification`, `recommendation`, `grading`, `risk`, `analytics` và `content`. Mỗi context gồm bốn lớp theo kiến trúc lục giác [12] (Bảng 2.2); một test kiến trúc kiểm tra quy tắc phụ thuộc giữa các lớp.

Bảng 2.2. Các lớp trong một bounded context

Lớp	Chứa gì	Được phụ thuộc vào
Lớp domain	Thực thể, value object, aggregate, sự kiện miền, *policy* (ví dụ `EngagementPolicy`, `MinCohortPolicy`)	Chỉ `shared_kernel.domain`
Lớp application	Use case (mỗi lớp một hàm `execute`), các cổng `Protocol`	domain
Lớp infrastructure	Adapter: Kafka, Redis, Delta, S3, Trino, OpenSearch, YAML	Mọi thứ
Lớp entrypoints	Điểm ghép nối cho job Spark, dịch vụ, API	Mọi thứ

Lợi ích cốt lõi là **một logic, nhiều engine**. Use case `NormalizeEvents` (kiểm tra hợp lệ, chuyển schema, khử định danh, gắn cờ trễ) chạy trong job streaming, job silver của lakehouse và dịch vụ xử lý sự kiện, cho cùng kết quả về bút danh và quyết định đưa vào DLQ. Các ngưỡng nghiệp vụ trong SQL mart lấy từ value object của domain và được test đơn vị giữ đồng bộ.

2.1.4 Ngữ nghĩa xử lý luồng

- **Thời gian sự kiện ở mọi nơi** theo mô hình Dataflow [13]. Watermark cho phép sự kiện đến muộn tối đa 5 phút với telemetry và 10 giây với CDC. Riêng cửa sổ rủi ro BR5 dùng watermark 21 ngày, vì dữ liệu học tập có thể được đồng bộ muộn.
- **Dữ liệu trễ không bị mất.** Sự kiện trễ hơn watermark vẫn được lưu và gắn cờ. Các model gold tự mở rộng cửa sổ tính toán về ngày của sự kiện trễ nhất (cơ chế `late_data`, Mục 3.1.8).
- **Đúng một lần.** Landing dùng checkpoint của Structured Streaming kết hợp sink giao dịch Delta. Lớp silver dùng MERGE idempotent theo khoá nghiệp vụ. Lớp phủ Redis ghi giá trị tuyệt đối thay vì cộng dồn, nên phát lại không đếm hai lần.
- **Thứ tự CDC.** Thay đổi được sắp theo `source.lsn` cho từng khoá. Thay đổi có LSN cũ hơn không thể ghi đè bản ghi mới hơn.

- **Đồng hồ thiết bị lệch.** Sự kiện có thời gian sau thời điểm tiếp nhận quá 2 phút được ghim về thời điểm tiếp nhận (`ClockSkewPolicy`). Nếu không, một thiết bị có đồng hồ chạy nhanh sẽ đẩy watermark lên và làm rơi sự kiện đúng hạn của mọi người khác.

2.1.5 Các quyết định kiến trúc

Mỗi quyết định quan trọng được ghi thành một ADR [14] trong bộ ADR của dự án. ADR đã chấp nhận không sửa; quyết định mới thay thế bằng ADR mới. Bảng 2.3 tóm tắt mười sáu ADR.

Bảng 2.3. Tóm tắt các quyết định kiến trúc (ADR)

ADR	Quyết định	Lý do và hệ quả chính
0001	Ghi quyết định bằng ADR	Tránh tranh luận lại; lưu lý do, không chỉ kết quả
0002	Monorepo Python, DDD + hexagonal	Use case độc lập engine; một chi phí nhỏ cho các lớp gián tiếp
0003	Kafka + Debezium cho tiếp nhận và CDC	Giữ thứ tự theo thực thể; phải giám sát replication slot
0004	Flink cho luồng, Spark cho lô	Lựa chọn ban đầu của nhóm; được điều chỉnh bởi ADR 0009 và 0014
0005	Lakehouse Delta Lake medallion	ACID upsert, time travel, ranh giới PII; cần job bảo trì chống file nhỏ
0006	Redis cho tầng phục vụ	Dưới mili-giây, TTL; Redis không phải nguồn sự thật
0007	Khử định danh ở ranh giới silver	HMAC-SHA256 có phiên bản khoá; báo cáo tổ chức áp dụng $k = 5$
0008	Runtime ban đầu trong tiến trình	Trình diễn đầu cuối trên laptop bằng adapter trong bộ nhớ; BR1–BR3 sau đó chuyển sang runtime thật (ADR 0016)
0009	PostgreSQL mart + Spark Structured Streaming cho BR4/BR5	Một driver Spark trên đường sản phẩm, chạy cả cửa sổ BR5 lẫn job tổng hợp BR4; job Flink tương đương chỉ dùng để so sánh engine; độ trễ CDC đến cảnh báo khoảng 10 giây (không đo lại)
0010	Trino + Hive Metastore làm SQL warehouse	Catalog và SQL warehouse trên chính các bảng Delta, không có bản sao thứ hai
0011	Spark là bên ghi duy nhất của Delta	Connector Flink cho Delta đã lỗi thời; tránh xung đột commit
0012	Gold là lược đồ hình sao Kimball, khoá tất định	`xxhash64` của khoá tự nhiên; dựng lại tái tạo đúng khoá; thành viên suy luận cho chiều đến muộn

ADR	Quyết định	Lý do và hệ quả chính
0013	Đồng bộ nội dung BR8 bằng CDC vào OpenSearch	Một đường đồng bộ cho mọi bên ghi; idempotent theo phiên bản đã chỉ mục
0014	Spark Structured Streaming cho telemetry nhanh và lớp phủ BR7/BR8	Chưa có job Flink cho telemetry và lớp phủ; độ trễ đo được khoảng 6 giây đủ cho lớp phủ
0015	Phiên đặc trưng BR2 trên Spark Structured Streaming + Redis	Spark 3.5 không hỗ trợ `update` cho `session_window`; state phiên nằm trong Redis, `SET NX` chống đếm lại
0016	Runtime thật cho BR1–BR3	Kafka, Redis, PostgreSQL thật; kết quả đến UI qua SSE; projector BR3 đồng bộ `public.*`

2.1.6 Điều chỉnh so với kiến trúc tham chiếu

Nhóm giữ nguyên vai trò của kiến trúc tham chiếu nhưng điều chỉnh ba điểm:

1. **Spark Structured Streaming thay Flink cho BR5 và lớp phủ BR7–BR8** (ADR 0009, 0014). Kiến trúc tham chiếu dùng Flink [15] cho xử lý luồng có trạng thái; khi hiện thực, job Flink mới ở mức khung còn Spark Structured Streaming đã chạy cho landing lakehouse. Một engine giảm hạ tầng vận hành và cho phép tái dùng code chuẩn hoá silver. Đổi lại, độ trễ khoảng 6 giây cho lớp phủ và khoảng 10 giây cho cảnh báo BR5 (số BR5 không đo lại), cao hơn mức dưới một giây của Flink, nhưng phù hợp với cảnh báo cho cố vấn và lớp phủ trên mart theo ngày. Về sau nhóm viết thêm một job Flink tính cùng các cửa sổ rủi ro (`infra/br45/flink/RiskWindows.java`) để so sánh engine cho BR5; job này chạy trong một stack benchmark riêng (`docker-compose.br45-benchmark.yml`), chỉ ghi vào bảng benchmark và không sinh cảnh báo, còn đường sản phẩm vẫn là Spark Structured Streaming. Cụm Flink vẫn có trong Compose chính, chưa chạy job nào, để chuyển đổi khi cần.
2. **Bổ sung Trino + Hive Metastore làm catalog và SQL warehouse** (ADR 0010). Delta Lake chỉ là định dạng bảng, nên kho dữ liệu cần thêm một catalog và một engine truy vấn. Hive Metastore và Trino đảm nhận hai vai trò này trên chính các bảng Delta, không tạo bản sao thứ hai, và chạy được cục bộ.
3. **BR2 dùng feature store hai nhánh.** Yêu cầu BR2 mô tả pipeline đặc trưng hai nhánh qua feature store; nhóm hiện thực cả hai: batch ghi `features.learner_longterm`, streaming giữ state phiên trong Redis. Gợi ý dùng feature làm ngữ cảnh và xếp hạng bằng

luật nghiệp vụ (Mục 4.1.2).

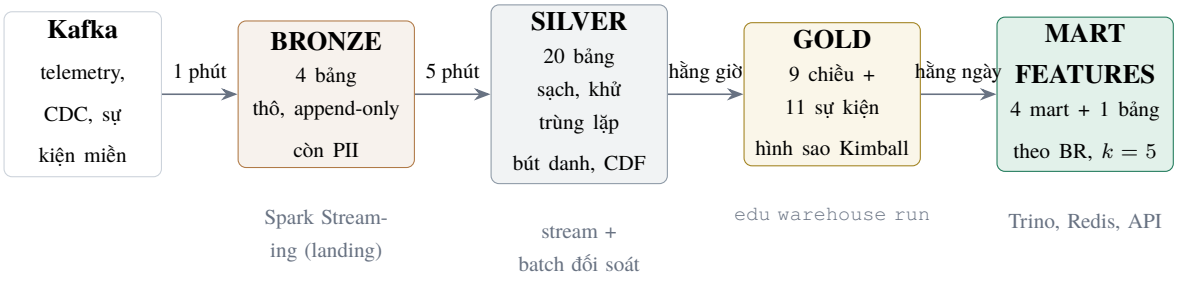
Chương 3

Lakehouse và kho dữ liệu

3.1 Tổng quan lakehouse

Chương này trình bày lakehouse [16] Delta Lake, tổ chức theo medallion [17], với kho dữ liệu chính là lớp gold. Nền tảng không có cơ sở dữ liệu phân tích thứ hai: Spark ghi bảng Delta, Hive Metastore lập danh mục, Trino phục vụ SQL và Redis xử lý tra cứu mili-giây. Toàn bộ DDL, model, cổng chất lượng và cấu hình nằm dưới dạng mã.

3.1.1 Kiến trúc medallion



Hình 3.1. Các lớp của lakehouse và tần suất cập nhật

Bảng 3.1. Hợp đồng dữ liệu của từng lớp

Schema	Số bảng	Bên ghi	Hợp đồng
`bronze`	4	Stream landing	Mỗi bản ghi Kafka một dòng, giữ nguyên như nhận được. Chỉ append, còn PII, chỉ vai trò `pii_steward` đọc được. Giữ 30 ngày (telemetry) hoặc 90 ngày (CDC, sự kiện). Phát lại bronze dựng lại được mọi lớp sau.
`silver`	19	Stream silver + batch đối soát	Có kiểu, đã kiểm tra hợp lệ, khử trùng lặp, đúng schema hiện hành, đã khử định danh bằng HMAC; bỏ định danh trực tiếp. CDC áp dụng theo thứ tự LSN, xoá mềm. Bật Change Data Feed.
`gold`	9 + 11	`edu warehouse run`	Lược đồ hình sao Kimball (ADR 0012): bảng chiều dùng chung, khoá thay thế tất định, bảng sự kiện có grain rõ ràng.
`mart`	4	`--layer mart`	Mỗi BR một bảng, theo nhu cầu người đọc; ngưỡng lấy từ policy của domain. `mart.org_progress` là bảng duy nhất quản trị viên B2B được đọc.
`features`	1	`--layer features`	Kho đặc trưng ngoại tuyến cho BR2, mỗi `as_of_date` một phân vùng để huấn luyện đúng thời điểm (point-in-time).
`ops`	2	Runner	`pipeline_runs` (mỗi lần build: cửa sổ, số dòng, phiên bản Delta) và `dq_results` (kết quả kiểm tra chất lượng).

3.1.2 Chuẩn hoá telemetry và tiến hoá schema

Telemetry đi qua use case `NormalizeEvents`, dùng chung cho job silver và job thời gian thực:

1. **Kiểm tra hợp lệ** theo hợp đồng schema: thiếu trường bắt buộc, verb ngoài từ vựng hoặc payload hỏng bị đưa vào DLQ kèm lý do.
2. **Chuyển schema**: `SchemaMigrator` nâng phiên bản 1.0 lên 1.1; phiên bản không nhận diện bị từ chối.
3. **Khử định danh**: thay định danh người học bằng bút danh; loại bỏ email, họ tên, IP và mã thiết bị.
4. **Thời gian**: tính độ trễ so với lúc tiếp nhận, gắn cờ `late` và ghim sự kiện có đồng hồ chạy

trước.

5. **Khử trùng lặp**: MERGE theo `event_id` (chỉ chèn) loại lần gửi lại từ thiết bị offline.

3.1.3 Khử định danh và phân quyền

Định danh thật chỉ tồn tại ở bronze (ADR 0007); từ silver trở đi, người học được biểu diễn bằng bút danh `k<version>_<24 hex>` sinh từ HMAC-SHA256 với khoá có phiên bản. Hàm SQL `pseudonymize()` dùng cùng thuật toán và khoá với `HmacPseudonymizer` của context telemetry, nên telemetry, CDC và sự kiện miền vẫn nối được. Cổng `silver_no_raw_identifiers` chặn định danh thô lọt qua bronze.

Trino áp dụng phân quyền chỉ đọc theo schema (Bảng 3.2). Quản trị viên B2B chỉ thấy đúng một bảng đã áp dụng k-anonymity.

Bảng 3.2. Phân quyền truy cập kho dữ liệu qua Trino

Người dùng	bronze	silver	gold	mart	features / ops
`pii_steward`	✓	✓	✓	✓	✓
`analyst`	–	✓	✓	✓	✓
`app` (API, dashboard)	–	–	✓	✓	–
`b2b_admin`	–	–	–	chỉ `org_progress`	–

3.1.4 Mô hình chiều Kimball ở lớp gold

Tám yêu cầu cùng đọc các sự kiện chồng lấn (học tập, bài nộp, ghi danh, rủi ro) theo những thực thể chung (người học, khoá học, nội dung, bài tập, tổ chức). Thiếu bảng chiều dùng chung, mỗi mart tự suy ra thực thể và dần lệch nhau; vì vậy lớp gold được mô hình hoá thành lược đồ hình sao Kimball [18] (ADR 0012). Bảng 3.3 là ma trận bus cho biết mỗi bảng sự kiện dùng chiều nào.

Bảng 3.3. Ma trận bus của lớp gold (D: vai trò ngày, •: khoá chiều)

Bảng sự kiện (grain)	*date*	*learner*	*course*	*content*	*activity*	*lesson*	*assignment*	*test case*	*org*	BR
`fct_learning_event` (một sự kiện)	D	•	•	•	•					6,7,8
`fct_learner_daily_activity` (người học × khoá × ngày)	D	•	•							2,8
`fct_video_second_activity` (phiên bản × giây × ngày)	D			•						7
`fct_submission` (một lần nộp)	D	•	•				•			3,4
`fct_test_case_result` (lần nộp × test)	D	•					•	•		4
`fct_lesson_progress` (người học × bài)		•	•			•				6
`fct_enrollment` (một lần ghi danh)	D×5	•	•						•	5,6
`fct_risk_score_daily` (người học × khoá × ngày)	D	•	•							5
`fct_dropout_alert` (một cảnh báo)	D	•	•							5
`fct_notification` (một kết quả gửi)	D	•								1
`fct_content_sync` (một phiên bản nội dung)			•	•						8

Bảng chiều và khoá thay thế

- **SCD loại 2** cho `dim_learner` và `dim_content`. Với `dim_learner`, báo cáo B2B phải quy hoạt động về tổ chức tại thời điểm xảy ra. Với `dim_content`, mỗi phiên bản video có một dòng riêng; video quay lại có độ dài khác nên giây thứ n của hai phiên bản không cùng nghĩa (BR7). Các chiều còn lại dùng SCD loại 1.
- **Khoá thay thế tất định**: `xxhash64(natural key)` với SCD1 và `xxhash64(natural key, valid_from)` với SCD2. Dựng lại một bảng chiều sẽ tạo lại đúng các khoá cũ, nên bảng chiều và bảng sự kiện build độc lập, không cần dịch vụ sinh số tuần tự.

- **Thành viên suy luận** (inferred member) cho chiều đến muộn: telemetry của một người học mà dòng CDC chưa đến sẽ tạo một dòng `is_inferred = true`, mang đúng khoá mà dòng thật sẽ có. Khi dòng thật đến, nó ghi đè tại chỗ nên không bảng sự kiện nào phải đổi khoá. Đã kiểm chứng: người học `l-131` có sự kiện hai ngày trước dòng CDC vẫn giữ nguyên khoá qua quá trình chuyển đổi.
- Khoá `-1` biểu diễn "không áp dụng", ví dụ nội dung của một sự kiện đăng nhập.

Một số quy tắc dùng chung: bài học hoàn thành khi CDC báo `percent_done = 100` *hoặc* telemetry có sự kiện `completed`, tuỳ sự kiện nào đến trước; vị trí video vượt độ dài phiên bản bị kẹp về giây cuối; lần chạy sandbox mới nhất thay thế các lần trước.

3.1.5 Data mart theo yêu cầu nghiệp vụ

Mỗi mart hiện thực một use case phân tích bằng SQL tập hợp. Ngưỡng được truyền từ value object của domain (`policy_params`), ghi trên từng dòng và có test đơn vị đối chiếu domain.

Bảng 3.4. Các data mart và kho đặc trưng

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`mart.assessment_item_stats`	BR4, giảng viên	Tỷ lệ đạt theo (bài tập, test case) trong 28 ngày; bất thường khi có ≥ 30 lượt, tỷ lệ đạt $\leq 0{,}35$ và thấp hơn trung vị của bài tập ít nhất $0{,}30$	`AnomalyThreshold`
`mart.org_progress`	BR6, quản trị B2B	Theo tổ chức × khoá học: quy mô nhóm, tỷ lệ hoàn thành, mức thành thạo trung bình; **nhóm dưới k người chỉ công bố quy mô** [19]	`MinCohortPolicy(k=5)`
`mart.video_heatmap`	BR7, giảng viên	Lưới dày theo từng giây cho mỗi phiên bản video trong 28 ngày; số người học phân biệt chính xác; cường độ 0–1	–
`mart.engagement_trend`	BR8, giảng viên	4 cửa sổ tuần cho mỗi người học × khoá; "giảm" khi điểm giảm $\geq 30\%$ trong mỗi cửa sổ của hai tuần gần nhất	`EngagementWeights`, tỷ lệ 0,30
`features.learner_longterm`	BR2, gợi ý	`ComputeBatchFeatures` trong 30 ngày, theo `as_of_date`	registry đặc trưng v1

BR1, BR3 và BR5 được phục vụ thời gian thực qua dịch vụ thông báo, dịch vụ chấm bài và runtime BR4–BR5 (ADR 0009). Kho dữ liệu lưu lịch sử phân tích bên cạnh: `fct_risk_score_daily` và `fct_dropout_alert` có nhãn `dropped_within_30d` để đánh giá độ chính xác cảnh báo; `fct_notification` và `fct_submission` phục vụ BR1 và BR3.

3.1.6 Thiết kế vật lý

- **Tính năng Delta**: ACID MERGE, `INSERT ... REPLACE WHERE` cho cửa sổ ngày, ghi đè phân vùng động, Change Data Feed ở silver, time travel (`FOR VERSION AS OF`) trong phạm vi VACUUM 7 ngày.
- **Phân vùng** chỉ theo cột ngày mà truy vấn lọc (ngày sự kiện, ngày hoạt động, ngày nộp, `as_of_date`); bảng nhỏ không phân vùng; bronze phân vùng theo `ingest_date`.
- **Gom cụm**: thuộc tính bảng `edu.zorder` được job bảo trì hằng ngày áp dụng bằng `OPTIMIZE ... ZORDER BY`. Cùng job này gộp các file nhỏ do writer streaming sinh ra.
- **Chính sách là thuộc tính bảng**: `edu.pii`, `edu.owner`, `edu.retention_days`, `edu.zorder` hiển thị cho mọi engine (`SHOW TBLPROPERTIES`, bảng `$properties` của Trino).
- **Tiến hoá schema tường minh**: tắt `autoMerge`; đổi DDL kèm câu lệnh migration.

3.1.7 Cổng kiểm tra chất lượng dữ liệu

Sau mỗi lớp, runner chạy bộ truy vấn kiểm tra chất lượng, ghi kết quả vào `ops.dq_results`; cổng mức `error` dừng lần chạy.

Bảng 3.5. Các cổng kiểm tra chất lượng

Cổng	Mức	Bảo vệ điều gì
`silver_event_id_unique`	error	Khử trùng lặp các lần gửi lại từ mobile
`silver_no_raw_identifiers`	error	Ranh giới PII: chỉ bút danh sau bronze
`bronze_dlq_rate`	warn	Hơn 5% telemetry bị từ chối
`gold_dim_learner_scd2`	error	Mỗi người học đúng một phiên bản hiện hành, không chồng lấn hiệu lực
`gold_orphan_keys`	error	Mọi khoá trong bảng sự kiện tồn tại trong bảng chiều
`gold_unresolved_learners`	warn	Sự kiện bị quy về thành viên "không rõ"
`mart_org_progress_k_anonymity`	error	Không công bố tỷ lệ cho nhóm dưới k người
`mart_rates_in_range`	error	Các tỷ lệ nằm trong $[0, 1]$
`mart_org_progress_unresolved_dims`	warn	Dòng báo cáo thiếu khoá học hoặc kỹ năng
`mart_video_heatmap_unversioned`	warn	Ô heatmap không gắn phiên bản video

3.1.8 Điều phối, dữ liệu trễ và chạy lại

Job streaming và dịch vụ chạy liên tục; job lô do Airflow điều phối (Bảng 3.6). Mọi lệnh lô idempotent theo `--run-date`, nên retry và backfill an toàn; `--start-date` dựng lại được bất kỳ khoảng thời gian nào.

Bảng 3.6. Các DAG Airflow

DAG	Lịch	Công việc
`medallion_daily`	hằng giờ	đối soát silver, build gold
`analytics_marts_daily`	02:00 hằng ngày	làm mới gold, mart BR4 / BR6 / BR7 cho ngày đã đóng
`engagement_weekly`	thứ Hai 02:00	`mart.engagement_trend`
`features_daily`	khi gold cập nhật	`features.learner_longterm`
`content_embedding`	6 giờ một lần	chia đoạn, nhúng học liệu
`lakehouse_maintenance`	04:00 hằng ngày	xoá theo thời hạn lưu, OPTIMIZE / Z-order, VACUUM

Bảng 3.7. Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự

Tình huống	Lớp	Cơ chế
Kafka gửi lại, job chạy lại	bronze	Checkpoint Structured Streaming + sink Delta (đúng một lần)
Client gửi lại (mobile offline)	silver	MERGE theo `event_id`; cổng `silver_event_id_unique`
CDC sai thứ tự hoặc bị phát lại	silver	Lấy thay đổi mới nhất theo `lsn`; chỉ MERGE khi `lsn` mới hơn; phát lại không tạo commit Delta
Xoá ở OLTP	silver	Debezium chế độ `rewrite`: `is_deleted = true` (xoá mềm)
Sự kiện trễ (mobile đồng bộ sau vài ngày)	gold	Model theo cửa sổ khai báo `late_data`; cửa sổ tự lùi về ngày sự kiện cũ nhất mà silver nhận kể từ lần build trước (đã kiểm chứng: lần chạy 06–14/10 tự mở rộng về 04/10)
Sự kiện đến trước chiều	gold	Thành viên suy luận với khoá cuối cùng (ADR 0012)
Xoá theo thời hạn lưu ở bronze	silver	`skipChangeCommits` để thao tác xoá không làm dừng stream

Tình huống	Lớp	Cơ chế
Tiến hoá schema của client	silver	Chuỗi `SchemaMigrator` 1.0 → 1.1; không biết thì vào DLQ

Khi có sự cố, bronze đóng vai nhật ký phát lại: sửa SQL rồi chạy lại `edu warehouse run --start-date` cho lớp bị ảnh hưởng; khi cần trạng thái cũ, dùng time travel của Delta với phiên bản lưu trong `ops.pipeline_runs`.

Chương 4

Hiện thực BR1–BR8 và ứng dụng

4.1 Thiết kế và hiện thực theo từng yêu cầu

Chương này trình bày tám yêu cầu nghiệp vụ. Với mỗi BR, báo cáo trình bày luồng dữ liệu, các quy tắc nghiệp vụ quan trọng và mức độ hoàn thiện. BR1–BR8 đều chạy trên hạ tầng thật: Kafka, Redis, PostgreSQL, Spark, Delta, Trino và OpenSearch. Dịch vụ notification và grading dùng Kafka, Redis, PostgreSQL; BR2 dùng job batch `features-online` và job streaming `telemetry-live` trên Delta và Redis.

4.1.1 BR1 — Pipeline thông báo hướng sự kiện

Luồng chạy thật. Dịch vụ `notification` là tiến trình độc lập, tiêu thụ năm topic Kafka: `lms.assignment_created.v1`, `lms.deadline_approaching.v1`, `lms.course_updated.v1`, `grading.submission_graded.v1` và `risk.dropout_risk_detected.v1`. Mỗi sự kiện đi qua rule engine, đối chiếu với tập luật YAML rồi phân giải người nhận từ PostgreSQL (`public.learners`, `public.enrollments`) hoặc cố vấn của học viên. Thông báo được ghi vào Redis và gửi qua in-app, push hoặc email. Kết quả được phát lên `notification.dispatched.v1` hoặc `notification.suppressed.v1` và ghi vào lakehouse thành `gold.fct_notification`. API đọc hộp thư từ Redis; cầu Kafka–SSE đẩy sự kiện `notification` xuống trình duyệt. Vòng tiêu thụ có retry/backoff và DLQ, nên một sự kiện lỗi không làm dừng dịch vụ.

Rule engine. Luật được khai báo trong YAML và nạp khi dịch vụ khởi động (Hình 4.1). Điều kiện viết bằng một ngôn ngữ biểu thức rất nhỏ trên payload của sự kiện, không cho chạy mã tuỳ ý.

```
rules:
  - rule_id: deadline-24h
    event_type: lms.deadline_approaching.v1
    condition: "hours_to_deadline <= 24"
    channels: [push, email]
    priority: high
    template_id: deadline_soon
  - rule_id: dropout-risk-advisor
    event_type: risk.dropout_risk_detected.v1
    channels: [email]      # recipient = the learner's advisor
    priority: high
```

Hình 4.1. Trích tập luật thông báo

Chống làm phiền. Mỗi thông báo có khoá khử trùng lặp trong Redis với TTL 24 giờ, nên cùng một sự kiện không gửi hai lần. Giới hạn tần suất dùng cửa sổ trượt 5 thông báo/giờ cho mỗi người học; thông báo ưu tiên cao (ví dụ còn dưới 24 giờ đến hạn) được miễn giới hạn. Kho thông báo idempotent theo định danh: hai use case định tuyến và gửi đều gọi `save()` nhưng chỉ lần tạo đầu thêm vào inbox và tăng bộ đếm, nên không nhân đôi thông báo. Thông báo bị chặn được ghi kèm lý do để giao diện hiển thị.

Kênh gửi thật. In-app ghi vào Redis; email đi qua SMTP; push đi qua webhook HTTP, cả hai có retry ba lần. Trong lần kiểm chứng, email và webhook đều đến đích; provider được cấu hình qua biến môi trường.

Đã kiểm chứng. Đăng ký `l-002` vào `c-ml-301` ghi một dòng trong PostgreSQL; sự kiện deadline tạo đúng một thông báo; phát lại cùng sự kiện trả lý do `duplicate` và không tăng `notif:count`. Trong cùng đợt kiểm chứng, email và webhook đều gửi thành công; Playwright xác nhận điều khiển gửi thông báo trên giao diện.

4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo

Yêu cầu BR2 mô tả pipeline đặc trưng hai nhánh: đặc trưng dài hạn tính theo lô bằng Spark, đặc trưng phiên tính theo luồng, qua feature store. Nhóm hiện thực cả hai và nối vào cùng online store Redis.

Nhánh lô. Bảng `features.learner_longterm` gồm tỷ lệ hoàn thành 30 ngày, tỷ lệ đạt trung bình, số ngày hoạt động, nền tảng ưa dùng và mức thành thạo theo kỹ năng. Mỗi ngày tính là một phân vùng, bảo đảm tính đúng theo mốc thời gian và không rò rỉ dữ liệu tương lai. Job `edu batch features-online` đọc phân vùng mới nhất và ghi `edu:features:longterm:<pseudonym>` vào Redis với TTL 36 giờ. Lần materialise ngày 2026-11-10 ghi 40 khoá từ bảng

Delta.

Nhánh luồng. Job `edu stream session-features` (chạy trong `telemetry-live`) đọc `telemetry.learning_events.v1`, giữ trạng thái phiên trong Redis và tách phiên khi khoảng trống quá 30 phút hoặc `session_id` đổi. Mỗi `event_id` được claim bằng `SET NX` trong 1 giờ, nên replay Kafka không đếm hai lần. Kết quả ghi `edu:features:session:<pseudonym>` với TTL 2 giờ và phát `features.session.v1` để lưu vết (ADR 0015). Micro-batch 30 giây giúp phiên đang mở hiển thị ngay, không phải chờ watermark.

Phục vụ. `LoadLearnerFeatures` kết hợp hai nhánh theo bút danh. API trả `features.longterm` và `features.session`; giao diện hiển thị chip "Live session" và "% recent pass". `NextStepPolicy` dùng feature làm ngữ cảnh: tỷ lệ đạt thấp hạ ưu tiên bước tiếp theo, phiên đang gặp khó ưu tiên bù kiến thức, cùng lộ trình thắng khi hoà. Khi ngữ cảnh rỗng, luật giữ nguyên hành vi cũ.

Bộ xếp hạng dựa trên **luật nghiệp vụ**, tính tại thời điểm truy vấn từ danh mục khoá học và các khoá đã hoàn thành. Luật xếp khoá chưa ghi danh theo *tầng*, rồi theo mức liên quan, độ khó và độ ưa thích (Bảng 4.1); mỗi gợi ý kèm một câu giải thích để người học hiểu vì sao.

Bảng 4.1. Các tầng của luật gợi ý (BR2)

Tầng	Trường hợp	Ví dụ lời giải thích	Điểm
0	Bước tiếp theo trong lộ trình	"Tiếp tục lộ trình data engineering: mức khó tiếp theo" hoặc "Bước liên quan trong lộ trình"	1,00
1	Mở rộng cùng mức	"Mở rộng ở mức của bạn: một khoá analytics mức dễ"	0,75
2	Bù khoá bị bỏ qua	"Bổ sung khoá mức dễ bạn đã bỏ qua"	0,50
3	Quá sức	"Quá sức: hoàn thành khoá mức trung bình trước" (khoá bị khoá)	0,00

Đã kiểm chứng. Năm sự kiện telemetry live cho `l-002` tạo phiên có 5 sự kiện; thêm 2 sự kiện và chạy lại thì phiên có 7 sự kiện, cùng `started_at`, còn replay không đếm lại. API trả cả `longterm` lẫn `session` sau khi build kho và materialise 40 khoá. Sáu unit test phủ policy, store, phép kết hợp và xếp hạng theo feature; Playwright xác nhận trang người học và danh sách gợi ý hiển thị.

4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết

Luồng tách rời, chạy thật. Giao diện gọi BFF `POST /api/submissions`; BFF chuyển tiếp sang `grading-ingest` ở cổng 8001. Dịch vụ ghi bài nộp `QUEUED` vào Redis và `grading.`

`submissions`, phát `grading.submission_received.v1`, chỉ trả 202 sau khi broker xác nhận (`publisher.flush()`); nếu queue không nhận, API trả 503 thay vì nhận việc rồi làm mất. Worker nhận message, claim `SET NX`, chuyển `RUNNING`, chạy sandbox và phát `grading.sandbox_result.raw.v1`. Normalizer áp `FeedbackPolicy`, ghi kết quả vào Redis và `grading.grades`, rồi phát `grading.submission_graded.v1`. Sự kiện này đi vào BR1, cầu SSE và projector đồng bộ `public.*`.

Vòng đời và tính idempotent. Trạng thái đi theo `queued` → `running` → `graded` hoặc `failed`. Worker claim theo `submission_id` nên message trùng không chạy sandbox lần hai; normalizer bỏ qua kết quả đã có theo `(submission_id, worker_run_id)`; PostgreSQL dùng UPSERT theo `submission_id`. Replay queue message hoặc raw result không tạo grade thứ hai.

Sandbox. Bài Python chạy trong tiến trình con có giới hạn thời gian 3 giây. Bài SQL chạy trên SQLite trong bộ nhớ ở chế độ chỉ đọc, có hàm ngắt khi quá hạn. Lỗi hạ tầng được tách khỏi lỗi trong mã của người học: lỗi và timeout của mã người học tính là trượt, còn lỗi hạ tầng không tính vào tỷ lệ đạt.

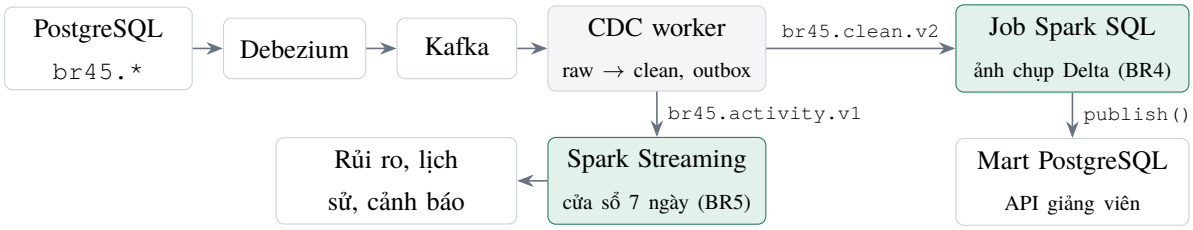
Phản hồi có cấu trúc. `FeedbackPolicy` là hàm thuần, không I/O, tạo phản hồi gồm câu tóm tắt, danh sách test trượt kèm giá trị mong đợi và thực tế (hoặc diff), 8 dòng cuối stderr và gợi ý theo ngôn ngữ dựa trên bảng mẫu lỗi thường gặp. Kết quả còn gồm thời gian chạy, bộ nhớ và mã thoát. Mục 4.2 trình bày cách hiển thị phản hồi trong Code lab.

Sở hữu dữ liệu. BR3 ghi `grading.submissions` và `grading.grades`; projector `grading-projector` đồng bộ sang `public.submissions`/`public.grades` theo profile `br13-project` để LMS và CDC nhận lịch sử.

Đã kiểm chứng. Nộp bài qua BFF trả 202 `queued`, sau đó chuyển `running` rồi `graded` với pass rate 0,8 và phản hồi "4/5 tests passed". Redis có đủ khoá submission, worker, result và code; PostgreSQL có dòng `graded` trong `grading.*`; hộp thư `l-002` có thông báo từ `grading.submission_graded.v1`; SSE phát `submission.graded`. Integration test end-to-end đạt; Playwright 2/2 đạt. Projector đồng bộ `public.*` từ 9/9 lên 11/11 trong lần kiểm chứng.

4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường

Luồng CDC thật (ADR 0009). Dữ liệu vận hành nằm trong schema `br45` của PostgreSQL: khoá học, bài tập, câu hỏi và test case có phiên bản, bài nộp và kết quả chấm. Connector Debezium riêng bắt WAL vào các topic `br45.cdc.br45.<table>`. Worker CDC lưu phong bì gốc vào `analytics.raw_changes`, kiểm tra hợp đồng (sai thì vào `quarantine`), khử trùng lặp theo định danh ổn định (bảng, LSN, giao dịch, thao tác, ảnh dòng) rồi ghi `clean_rows`. Thay đổi có LSN cũ không thể ghi đè bản ghi mới hơn. Trong cùng giao dịch, mỗi dòng được chấp nhận còn được ghi vào outbox `analytics.domain_outbox` để worker phát lên topic `br45.clean.v2`.



Hình 4.2. Đường CDC thật của BR4 và BR5

Ngữ nghĩa chỉ số. Mọi khoảng quan sát là nửa mở $[t_0, t_1)$. Một lần chấm test hợp lệ khi bài nộp đã được chấm, kết quả hợp lệ và trạng thái thuộc {passed, failed, error, timeout}. Một lần làm câu hỏi hợp lệ khi mọi test bắt buộc của phiên bản câu hỏi *tại thời điểm nộp* có kết quả hợp lệ; lần làm đó đạt khi mọi test bắt buộc đều đạt. Phiên bản test mới hơn không diễn giải lại bài nộp cũ.

$$\text{pass_rate} = \frac{\text{số lần chấm hợp lệ đạt}}{\text{tổng số lần chấm hợp lệ}}$$

Mẫu số bằng 0 cho kết quả `null` (hiển thị là chưa có số liệu). Hệ thống cũng tính tỷ lệ đạt ở lần làm đầu tiên và lần làm cuối cùng của mỗi học viên.

Bảng 4.2. Quy tắc phát hiện bất thường của BR4 (cấu hình được)

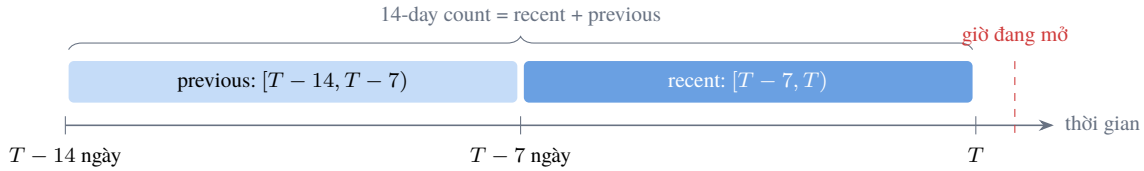
Quy tắc	**Mặc định**	**Trạng thái sinh ra**
Số học viên hợp lệ phân biệt tối thiểu	20	`INSUFFICIENT_DATA` nếu thiếu
Tỷ lệ đạt thấp của câu hỏi	dưới 30%	`LOW_PASS_RATE`
Tỷ lệ đạt thấp của test case	dưới 20%	`LOW_PASS_RATE`
Thấp hơn trung vị các test bắt buộc khác	$\geq$ 50 điểm %	`SUSPICIOUS_TEST_CASE`
Giảm so với cửa sổ liền trước cùng độ dài	$\geq$ 30 điểm %	`NEEDS_REVIEW`

Kết quả lưu trong ba mart của PostgreSQL (`mart_question_pass_statistics`, `mart_test_case_pass_statistics`, `mart_question_anomalies`) kèm định danh, phiên bản, cửa sổ, số đếm, lý do và ngưỡng. Việc tổng hợp chạy bất đồng bộ (Hình 4.2). Khi một thay đổi CDC của dữ liệu chấm bài được chấp nhận, hoặc khi mốc đánh giá tiến lên, worker CDC ghi một job bền vào `analytics.assessment_jobs` cho cửa sổ 30 ngày của từng khoá học có bài nộp. Job br45-stream nạp các dòng từ `br45.clean.v2` vào một bảng Delta bằng MERGE theo khoá, chỉ nhận dòng có LSN mới hơn. Khi bảng này đã chứa thay đổi mà job yêu cầu, br45-stream tổng hợp bằng Spark SQL trên một ảnh chụp có phiên bản của bảng, rồi gọi `publish()`: sau một cổng chất lượng, toàn bộ dòng mart của khoá học và cửa sổ đó được thay trong một giao dịch PostgreSQL. Vì mart được thay bằng giá trị tuyệt đối theo khoá ổn định thay vì cộng dồn, sửa điểm hay xoá

bản ghi được phản ánh đúng mà không cần chạy lại seed. API chỉ đọc mart và báo trạng thái của job, không tự chạy phép tổng hợp. Phép tổng hợp bằng SQL của PostgreSQL trên `clean_rows` vẫn được giữ làm engine đối chứng, chọn bằng `EDU_BR45_ASSESSMENT_ENGINE=postgres`. Một cờ bất thường chỉ yêu cầu giảng viên xem xét, không kết luận đề bị lỗi.

4.1.5 BR5 — Cảnh báo sớm nguy cơ bỏ học

Cửa sổ trượt theo thời gian sự kiện. Spark Structured Streaming đọc `br45.activity.v1`, dùng watermark 21 ngày và `dropDuplicatesWithinWatermark(event_id)`, rồi tổng hợp bằng `window(event_time, '7 days', slide)` với bước trượt mặc định 1 giờ. Đây là tổng hợp cửa sổ có trạng thái, không phải truy vấn SQL chạy theo lịch. Với mốc đánh giá T theo giờ UTC, các đặc trưng được tính như Hình 4.3.



Hình 4.3. Các cửa sổ đặc trưng tại mốc đánh giá T (BR5)

Cập nhật một hoạt động sẽ rút phần đóng góp cũ khỏi các cửa sổ rồi cộng phần mới; xoá thì rút phần đóng góp. Sự kiện "đồng hồ" trọng số 0 tạo cửa sổ rỗng để ghi nhận tình trạng không hoạt động. Mỗi micro-batch upsert tổng tuyệt đối của cửa sổ trong một giao dịch. Điểm rủi ro chỉ được chấm khi offset Spark đã commit phủ hết outbox, tránh cảnh báo trên đặc trưng chưa đầy đủ.

Điều kiện xét. Học viên phải có ghi danh hiệu lực ít nhất 7 ngày. Bài tập bắt buộc chỉ tính khi hạn nộp nằm trong 14 ngày gần nhất và sau ngày ghi danh; hạn tương lai không bị phạt. Thiếu telemetry không bị coi là không hoạt động: tín hiệu này chỉ hợp lệ khi từng quan sát thấy hoạt động thật.

Bảng 4.3. Các tín hiệu rủi ro và điểm (BR5)

Tín hiệu	Điều kiện	Điểm
Không hoạt động	Ít nhất 7 ngày kể từ hoạt động có ý nghĩa gần nhất	35
Trễ hạn	Ít nhất một bài tập bắt buộc đã đến hạn mà chưa nộp	30
Tỷ lệ nộp thấp	Tỷ lệ nộp các bài đã đến hạn dưới 50%	20
Hoạt động giảm	Hoạt động giảm ít nhất 50%, với ít nhất 3 quan sát ở cửa sổ trước	15

Điểm tối đa là 100, chia mức LOW 0–29, MEDIUM 30–59, HIGH 60–79 và CRITICAL 80–100. Mọi kết quả kèm đặc trưng, giải thích, điều kiện xét, thời điểm đánh giá và chính sách. **Điểm không phải xác suất bỏ học**; đây là công cụ giúp cố vấn ưu tiên.

Vòng đời cảnh báo. Mỗi cặp học viên × khoá học chỉ có một cảnh báo mở. Vượt 60 điểm sinh cảnh báo mới; thời gian chờ mặc định 72 giờ. Cảnh báo đang mở được cập nhật mức độ và lý do. Cố vấn có thể xác nhận (acknowledge) hoặc giải quyết (resolve); hệ thống lưu thời điểm và người thực hiện. Khi điểm xuống dưới ngưỡng, cảnh báo tự động giải quyết. Lịch sử mọi lần đánh giá và sửa đổi được giữ để kiểm toán.

4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức

BR6 là ví dụ đầy đủ nhất của pipeline medallion. Telemetry và CDC đi qua silver (chuyển schema, khử định danh), lên gold (`fct_enrollment`, `fct_lesson_progress`, `fct_submission`) và tổng hợp thành `mart.org_progress` theo tổ chức × khoá học. Tỷ lệ hoàn thành là trung bình tỷ lệ bài học đã xong của khoá; mức thành thạo là tỷ lệ đạt test trung bình của các bài nộp đã chấm, học viên chưa có bài chấm tính là 0.

k-anonymity được bảo vệ ở ba lớp. (1) Mart chỉ công bố quy mô cho nhóm dưới $k = 5$ người; các tỷ lệ để trống. (2) Cổng chất lượng mức `error` `mart_org_progress_k_anonymity` làm hỏng lần build nếu vi phạm. (3) Bộ đọc API xoá tỷ lệ của dòng bị ẩn, kể cả khi mart lỗi vẫn mang giá trị. Quản trị viên chỉ đọc được tổ chức trong phạm vi tài khoản của mình; truy cập ngoài phạm vi trả 403.

4.1.7 BR7 — Heatmap tương tác video

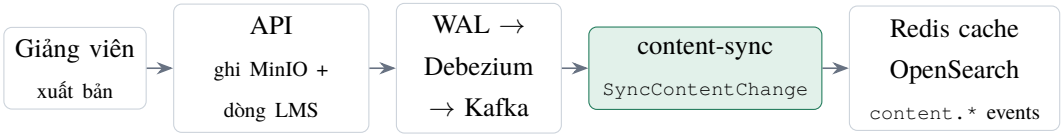
Theo phiên bản. Sự kiện được quy về phiên bản nội dung có hiệu lực tại thời điểm xảy ra (`gold.dim_content` là SCD2). Video quay lại có heatmap riêng cho mỗi phiên bản; API mặc định chọn phiên bản mới nhất. Bảng `gold.fct_video_second_activity` có grain phiên bản × giây × ngày, đếm play, pause, seek và số người học phân biệt. `mart.video_heatmap` tạo lưới dày từng giây trong 28 ngày, với số người học phân biệt chính xác và cường độ chuẩn hoá 0–1. Vị trí vượt độ dài video được kẹp về giây cuối.

Lớp phủ hôm nay (ADR 0014). `telemetry-live` chạy ba truy vấn Spark có checkpoint. Truy vấn thứ nhất chuẩn hoá telemetry thô bằng `normalize_rows` của silver; hai truy vấn còn lại tổng hợp theo giây video (BR7) và điểm tương tác theo ngày (BR8), dùng watermark 5 phút và khử trùng lặp theo `event_id`. Kết quả ghi vào Redis dưới dạng giá trị tuyệt đối, nên replay ghi đè thay vì cộng thêm. API chỉ ghép lớp phủ cho ngày sau ảnh chụp mart và cho phiên bản mới nhất; nếu Redis lỗi, API trả dữ liệu mart.

4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác

4.1.8.1 Đồng bộ cache và chỉ mục tìm kiếm

Một phiên bản nội dung mới tương ứng lần cập nhật dòng `public.course_content`, trỏ tới đối tượng bất biến mới trong MinIO (`courses/<course>/<content>/v<n>.<ext>`). API giảng viên chỉ ghi đối tượng và dòng dữ liệu dưới khoá dòng, tránh hai người xuất bản cùng lúc; việc đồng bộ do `content-sync` đảm nhận. Debezium bắt thay đổi; dịch vụ `content-sync` chạy use case `SyncContentChange` (Hình 4.4).



Các bước trong content-sync:
1. Lấy đối tượng 2. Vô hiệu hoá cache Redis 3. Chia đoạn + nhúng vector 4. Dọn phiên bản khác
5. Làm mới mục tìm kiếm 6. Làm ấm cache 7. Phát sự kiện

Hình 4.4. Đồng bộ nội dung BR8 dựa trên CDC

Idempotent theo trạng thái chỉ mục, không theo thứ tự đến. `sync_policy` chỉ áp dụng phiên bản mới hơn phiên bản đã chỉ mục; bằng nhau là trùng lặp, cũ hơn là lỗi thời. Vì vậy replay CDC, khởi động lại hay backfill đều an toàn. Tìm kiếm chỉ trả đoạn thuộc phiên bản hiện hành, nên dọn dẹp chậm không làm lộ văn bản cũ.

Tìm kiếm lai. OpenSearch lưu đoạn văn bản với chỉ mục BM25 và vector `knn_vector` (HNSW [20]); kết quả hai phía được hợp nhất bằng reciprocal rank fusion [21] ở client. Vector hiện do bộ nhúng băm tất định sinh (không cần mô hình hay GPU), nằm sau cổng `Embedder`; thay bằng sentence-embedding chỉ cần adapter mới và một lần reindex. Job Spark `edu batch content-reindex` phân phối *cùng* use case theo nhóm nội dung để backfill và ghi `silver.content_chunks`.

4.1.8.2 Xu hướng tương tác

Điểm tương tác của mỗi học viên trong một cửa sổ là tổng có trọng số các hoạt động (Bảng 4.4). Mart `mart.engagement_trend` tính bốn cửa sổ tuần (thứ Hai đến Chủ nhật) cho mỗi cặp học viên × khoá học.

Bảng 4.4. Trọng số hoạt động trong điểm tương tác (`EngagementWeights`)

Nhóm	Động từ (verb)	Trọng số
Đăng nhập	`logged_in`	1,0
Xem bài	`viewed`, `played`, `completed`	1,0
Làm bài	`attempted`, `submitted`	2,0
Thảo luận	`posted`	1,5

Gọi a, b, c là điểm của ba cửa sổ gần nhất theo thứ tự thời gian, và $r = 0{,}30$ là tỷ lệ giảm. Học viên được đánh dấu *đang giảm* khi

$$a > 0 \ \wedge \ b \leq a\,(1 - r) \ \wedge \ c \leq b\,(1 - r),$$

tức là điểm giảm ít nhất 30% trong *mỗi* cửa sổ của hai tuần liên tiếp. Lớp phủ "hôm nay" dùng cùng trọng số, nên giá trị trong ngày khớp với giá trị kho dữ liệu tính vào hôm sau.

Pipeline chỉ có giá trị khi kết quả đến được người dùng. Mục 4.2 trình bày ứng dụng web cho năm nhóm: người học, giảng viên, cố vấn học tập, quản trị viên tổ chức và đội vận hành nền tảng dữ liệu.

4.2 Giao diện người dùng

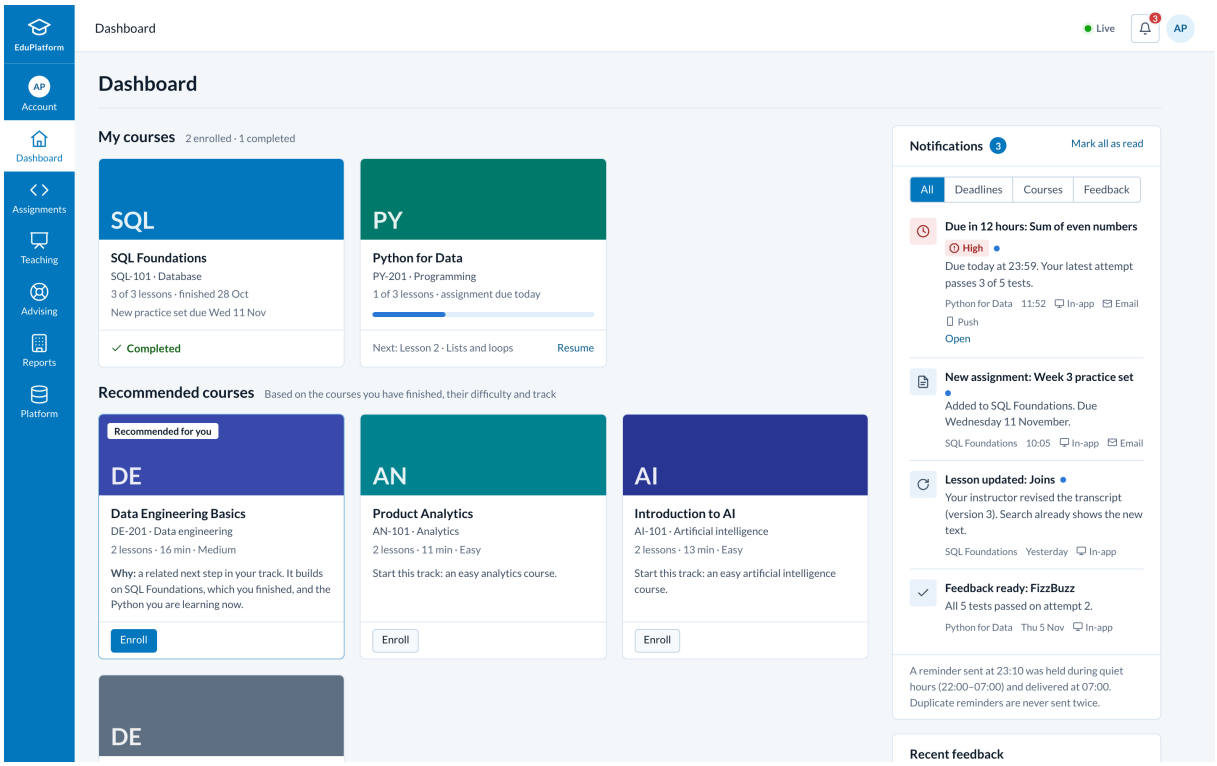
4.2.1 Quy trình thiết kế

Giao diện trải qua ba vòng thiết kế:

1. **Bảng điều khiển hướng nhà phát triển**: nút giả lập sự kiện, mã định danh thô và ô nhập token nằm lẫn với kết quả. Bản này chứng minh được API nhưng khó dùng với người dùng cuối.
2. **Mockup HTML tương tác và bản React đầu tiên**: mockup dùng dữ liệu mẫu nhưng tuân theo quy tắc nghiệp vụ thật (điểm rủi ro 35/30/20/15, ngưỡng 30/60/80, $k = 5$, tỷ lệ giảm 30%), nhờ đó thống nhất bố cục trước khi viết mã. Khi xem lại, nhóm nhận thấy bản này mang dáng một dashboard chung chung (thẻ bo tròn lớn, nền chuyển màu, nhãn in hoa) hơn là một hệ thống quản lý học tập.
3. **Thiết kế lại theo quy ước LMS**: dựa trên Canvas và Moodle [22, 23], dùng thanh điều hướng toàn cục bên trái, menu theo khu vực, breadcrumb, thẻ khoá học có dải màu và trang bài tập có dòng thông tin chi tiết; chế độ sáng tông xanh da trời là mặc định. Mockup dùng

chung stylesheet với ứng dụng nên hai bản luôn khớp nhau (Hình 4.5).

Ứng dụng viết bằng React 18 + TypeScript trên Vite, không thêm thư viện giao diện hay biểu đồ; mọi màn hình đọc API thật. Phần mockup không có API tương ứng (độ tươi báo cáo, kết quả kiểm tra chất lượng, ghi chú can thiệp) được bỏ thay vì dùng dữ liệu giả.



Hình 4.5. Mockup: Dashboard người học với thẻ khoá học (BR2), thông báo (BR1) và phản hồi gần đây (BR3)

4.2.2 Đáp ứng yêu cầu về giao diện

Đề bài yêu cầu kết quả của mỗi BR hiển thị phù hợp với ngữ cảnh miền ứng dụng, với đặc điểm và thói quen của nhóm người dùng đích, đồng thời hài hoà trong trình bày lẫn tương tác.

Ngữ cảnh miền ứng dụng. Người học và giảng viên đã quen với các LMS như Canvas và Moodle, nên ứng dụng dùng lại bố cục và từ vựng của chúng thay vì đặt ra một kiểu điều hướng mới. Cụ thể: thanh điều hướng toàn cục màu xanh da trời ở cạnh trái, mỗi mục gồm biểu tượng và nhãn (Account, Dashboard, Assignments, Teaching, Advising, Reports, Platform); menu chữ theo khu vực, ẩn hoặc hiện bằng nút ba gạch; breadcrumb cho biết vị trí hiện tại, ví dụ “Teaching › Assessment health”. Dashboard của người học đặt thẻ khoá học có dải màu, mã khoá và tiến độ; khoá đề xuất (BR2) hiện thành thẻ có nhãn “Recommended for you” và dòng “Why” giải thích lý do; cột phải chứa Notifications (BR1) và Recent feedback (BR3), tương tự khối To Do và Recent Feedback của Canvas. Trang bài tập (BR3) theo mẫu trang Assignment: tên khoá học,

tên bài, dòng "Language · Tests · Attempts · Submitting", phần Instructions, khung phản hồi và Submission history. Nhãn BR1–BR8 phục vụ truy vết khi trình bày nhưng không có trong LMS thật, nên mặc định tắt và bật lại được ở Account › Preferences.

Đặc điểm người dùng đích. Mỗi nhóm có thiết bị, nhịp làm việc và câu hỏi riêng; Bảng 4.5 ánh xạ các đặc điểm đó vào lựa chọn thiết kế cụ thể.

Bảng 4.5. Đặc điểm người dùng và lựa chọn thiết kế tương ứng

Nhóm	Đặc điểm, thói quen	Lựa chọn thiết kế
Người học	Học trên cả điện thoại lẫn máy tính, phiên ngắn; cần biết ngay việc tiếp theo và lý do của mỗi gợi ý	Thẻ khoá học, hạn nộp và thông báo trên cùng một trang; gợi ý luôn kèm lý do; bố cục co về 390 px, thanh bên thành dải ngang; trang bài tập tách rõ đề bài, mã và phản hồi từng test
Giảng viên	Làm việc trên máy tính, quản lý nhiều khoá; xem phân tích theo từng khoá và từng bài	Trang phân tích nằm trong khu Teaching có menu riêng; bộ lọc khoá học ở đầu trang; bảng có hàng chọn được để xem chi tiết; heatmap đánh dấu A, B, C các thời điểm cần xem lại
Cố vấn	Xử lý hàng đợi theo mức ưu tiên; cần biết vì sao một người học bị gắn cờ trước khi liên hệ	Hàng đợi xếp theo điểm rủi ro; điểm phân rã thành 35 + 30 + 20 + 15 kèm câu giải thích; nút Acknowledge và Resolve theo vòng đời cảnh báo
Quản trị tổ chức	Đọc báo cáo định kỳ, chuyển tiếp cho cấp trên; quan tâm quyền riêng tư	Ô tổng hợp, bảng theo khoá học và nút Export CSV; nhóm dưới 5 người bị ẩn kèm thông báo giải thích, không hiện tỷ lệ hay quy mô chính xác
Đội nền tảng	Theo dõi sức khoẻ pipeline trong lúc vận hành	Bộ đếm từng chặng ingest → bronze → silver → đầu ra, hàng đợi dead-letter và luồng sự kiện trực tiếp

Hài hoà trong trình bày và tương tác. Một màu nhấn duy nhất là xanh da trời `#0277BD`, dùng cho thanh điều hướng, liên kết, nút chính và mục đang chọn; dải màu tuần tự của heatmap và chuỗi dữ liệu thứ nhất của biểu đồ cũng thuộc họ xanh dương, nên biểu đồ và khung giao diện cùng tông. Chế độ sáng là mặc định, kể cả khi hệ điều hành đang để chế độ tối, vì người dùng LMS thường đọc lâu và học vào ban ngày; chế độ tối là tuỳ chọn trong Account › Preferences, dùng nền xanh navy với cùng màu nhấn. Phong cách phẳng và nhất quán: font Lato, góc bo 4–6 px, khối nội dung viền mảnh, không bóng đổ, không nền chuyển màu.

Màu trạng thái (tốt, cảnh báo, nghiêm trọng, nguy hiểm) chỉ dùng cho trạng thái và luôn kèm

biểu tượng cùng nhãn chữ; bảng màu biểu đồ đã kiểm định khả năng phân biệt với người mù màu trên cả nền sáng lẫn nền tối; các cặp chữ và nền chính đạt tương phản tối thiểu 4,5:1 theo WCAG 2.1 AA [24], kể cả chữ trắng trên dải màu của thẻ khoá học. Tương tác theo một mẫu chung ở mọi trang: bộ lọc ở đầu trang, các ô tổng hợp, danh sách hoặc bảng, rồi chi tiết của hàng đang chọn; hàng được chọn có nền nhạt và vạch màu nhấn bên trái, mọi điều khiển có viền focus khi dùng bàn phím.

4.2.3 Hệ thống thiết kế

- **Token thiết kế** cho màu, khoảng cách, bo góc và bóng đổ, có bộ giá trị riêng cho chế độ sáng và tối; chế độ sáng là mặc định, chế độ tối chuyển ở Account › Preferences.
- **Khả năng tiếp cận**: mọi biểu đồ có tooltip và bảng "Xem dạng bảng" thay thế; nút có nhãn cho trình đọc màn hình; bố cục co giãn tới điện thoại mà không cuộn ngang.
- **Nhãn yêu cầu**: mỗi trang và khối được gắn nhãn BR, mặc định tắt và bật lại được ở Account › Preferences để truy vết giao diện về yêu cầu khi trình bày.

4.2.4 Kiến trúc frontend

Ứng dụng tổ chức theo *workspace*, mỗi nhóm người dùng một workspace chọn ở thanh bên (Bảng 4.6). Thanh trên cùng có chỉ báo sự kiện trực tiếp (Server-Sent Events), thông báo người học và trạng thái đăng nhập; mục Account mở khay cho biết vai trò đang đăng nhập và chứa tuỳ chọn chế độ tối cùng nhãn yêu cầu.

Bảng 4.6. Workspace, đường dẫn và trang

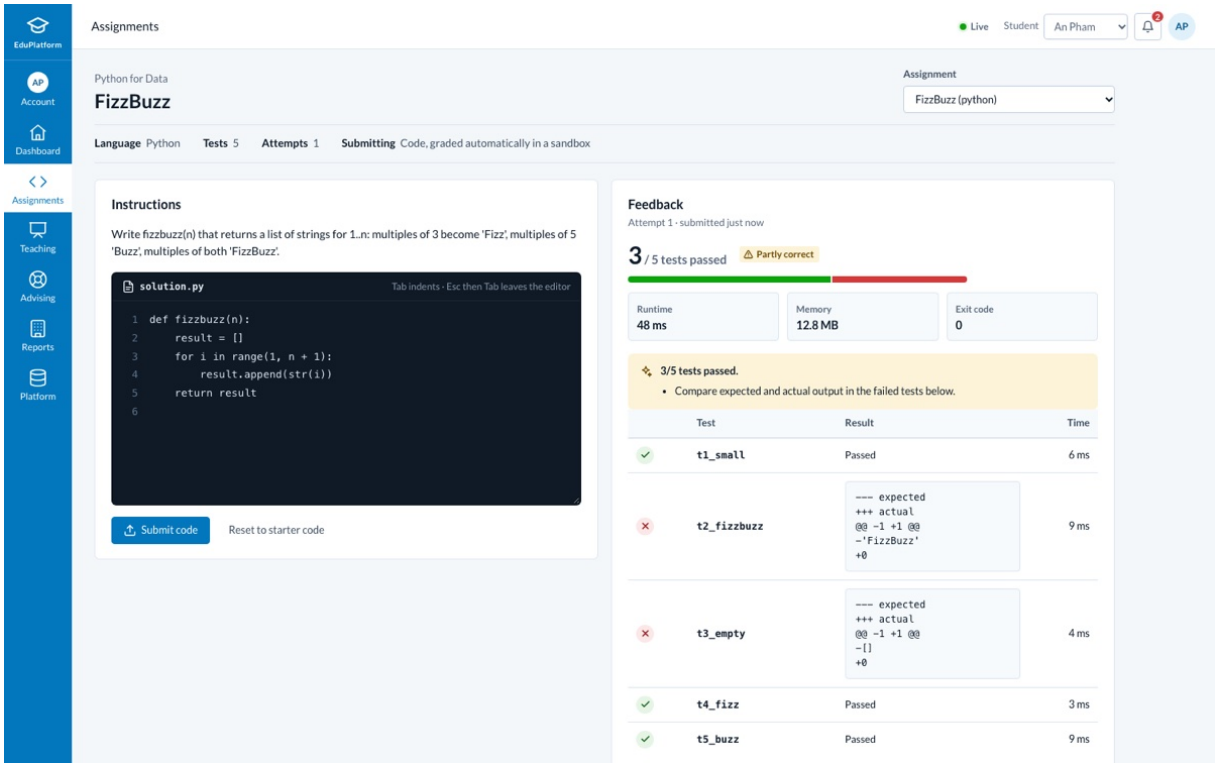
Workspace	Đường dẫn	Nội dung	BR
Learner	`/learner`	Khoá học, gợi ý, phản hồi gần đây, thông báo	1, 2
Learner	`/learner/code-lab`	Trình soạn mã, phản hồi từng test, các lần nộp	3
Instructor	`/instructor/assessments`	Câu hỏi và chi tiết test case	4
Instructor	`/instructor/videos`	Heatmap theo phiên bản, lớp phủ hôm nay, thời điểm cần xem lại	7
Instructor	`/instructor/engagement`	Trung bình khoá học, học viên đang giảm	8

Workspace	Đường dẫn	Nội dung	BR
Instructor	`/instructor/content`	Xuất bản phiên bản, theo dõi đồng bộ, xem trước tìm kiếm	8
Advisor	`/advisor`	Hàng đợi rủi ro, phân rã điểm, lịch sử, cảnh báo	5
Org admin	`/admin`	Tiến độ tổ chức có ẩn nhóm nhỏ, xuất CSV	6
Data platform	`/pipeline`	Bộ đếm luồng, thông lượng, topic, DLQ, silver	–

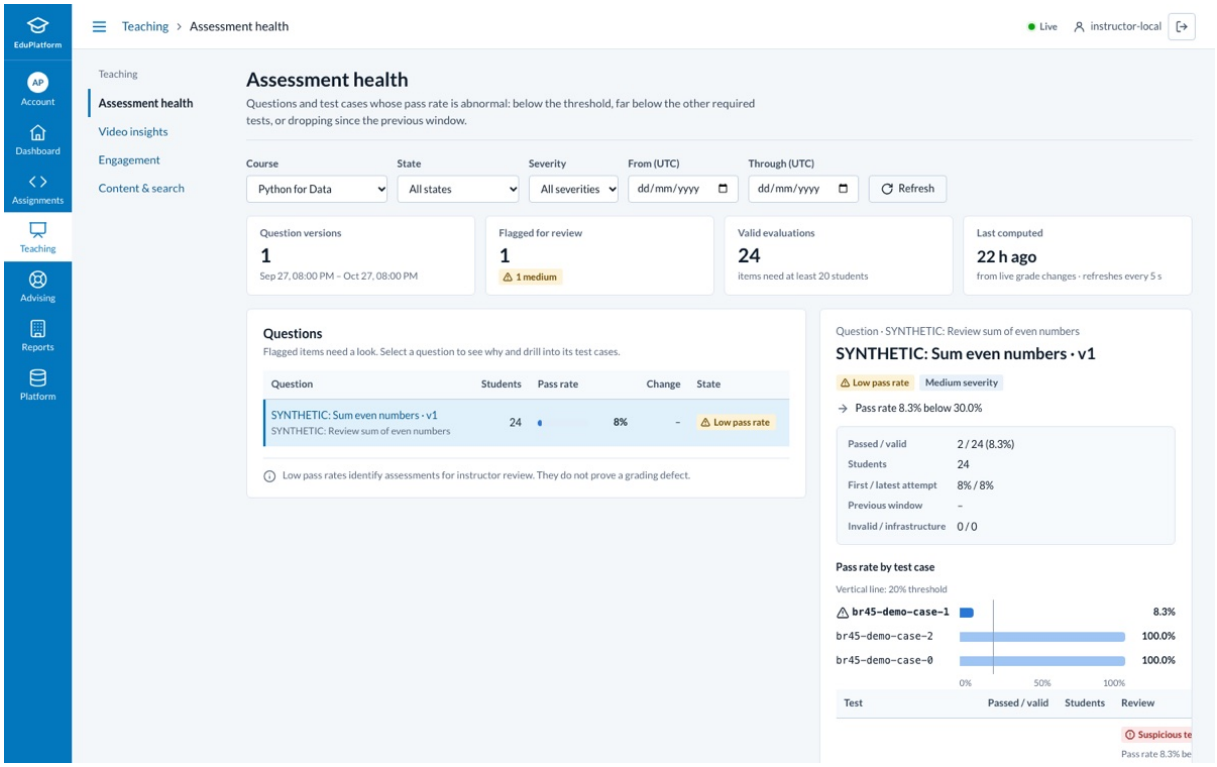
Workspace Instructor, Advisor và Org admin được bảo vệ bằng thẻ đăng nhập bằng tên người dùng và mật khẩu, mỗi workspace một phiên riêng. Máy chủ đối chiếu mật khẩu với mã băm Argon2id của các tài khoản cấu hình sẵn (`EDU_STAFF_ACCOUNTS`) rồi cấp cookie phiên HttpOnly theo vai trò, có hạn tám giờ; các yêu cầu sau đó gửi cookie này kèm header `X-Staff-Role`. Danh tính, vai trò và phạm vi khoá học/tổ chức lấy từ tài khoản phía máy chủ, giao diện không tự khai báo quyền. Các điều khiển thử nghiệm (giả lập sự kiện LMS, bộ sinh telemetry) được tách vào khối “Demo controls” viền nét đứt. Biểu đồ vẽ bằng SVG thuần theo chiều rộng thật của phần tử, nên chữ không bị co giãn.

4.2.5 Các màn hình chính

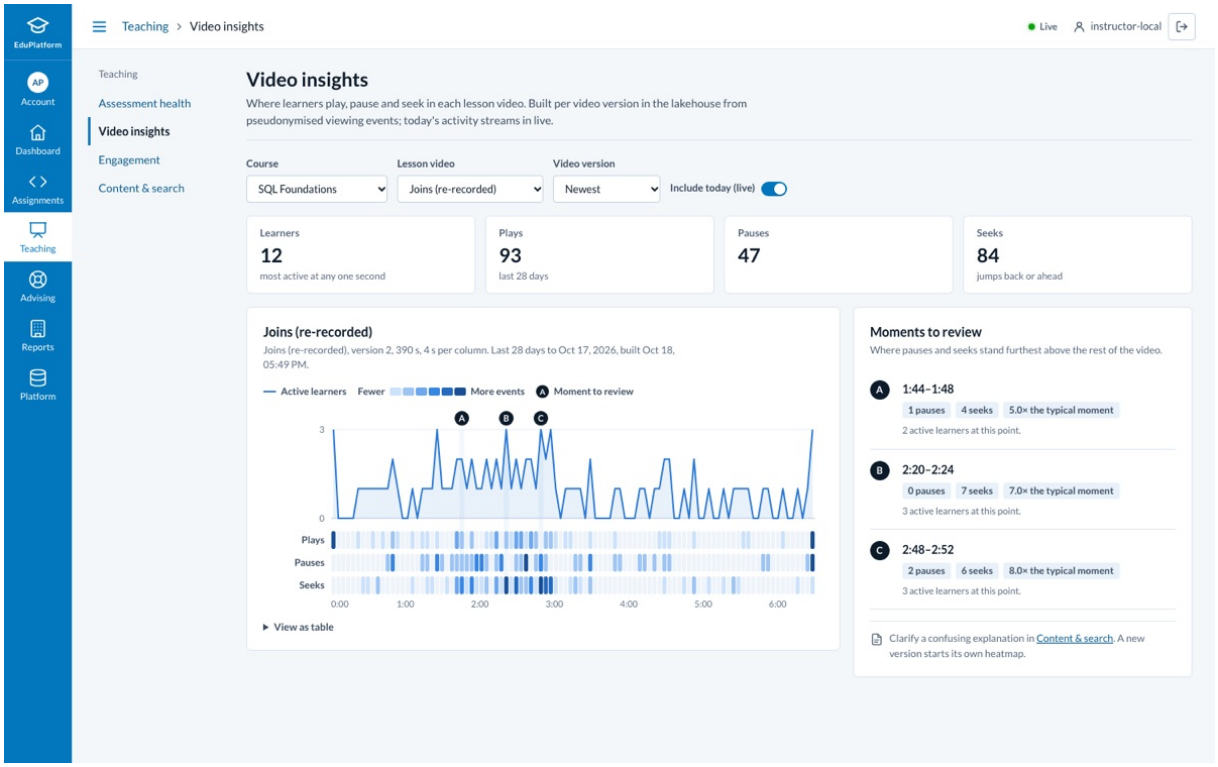
Ảnh trong mục này chụp từ ứng dụng thật trên dữ liệu hệ thống, ở chế độ sáng mặc định, trừ khi ghi là mockup.



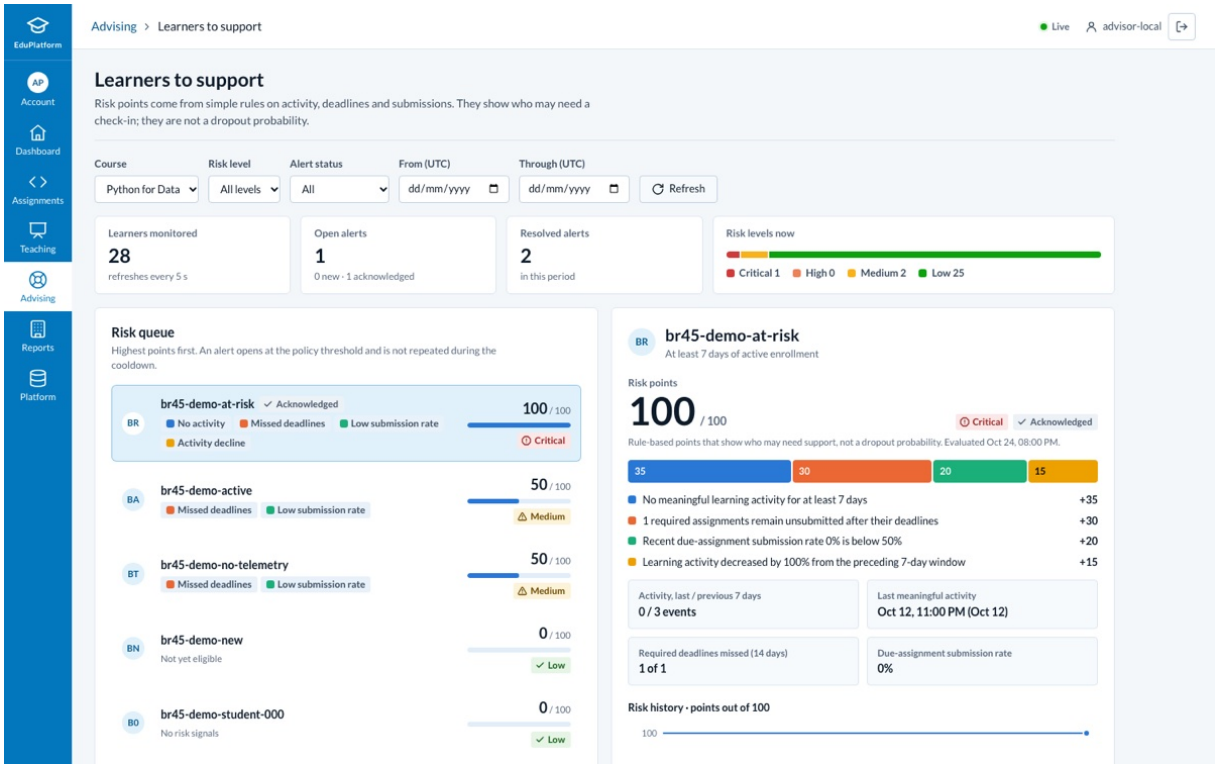
Hình 4.6. Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff



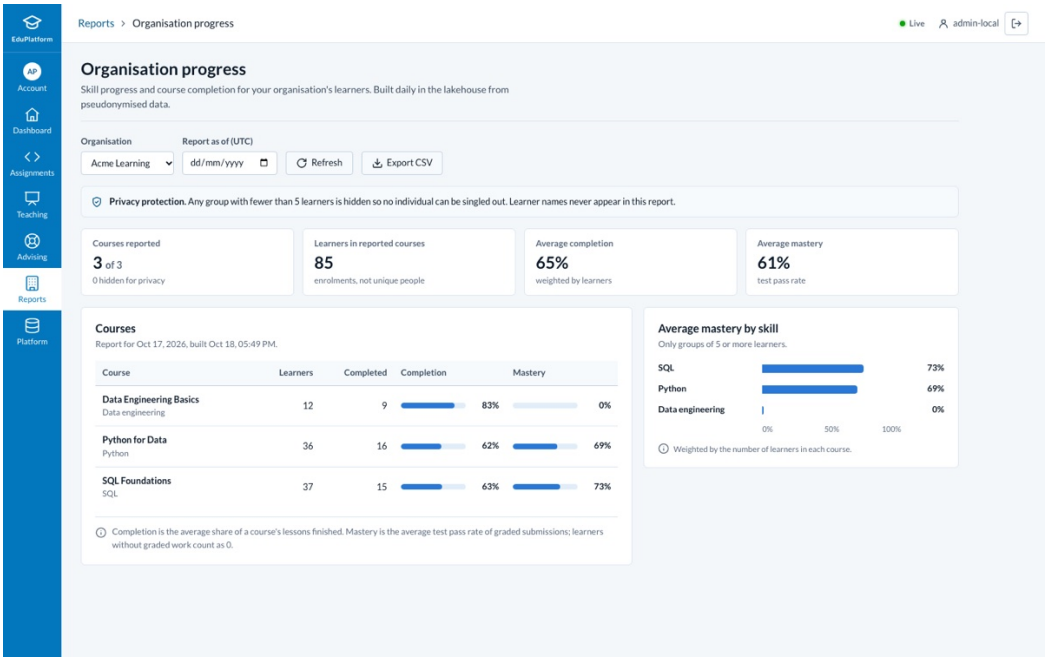
Hình 4.7. Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20%



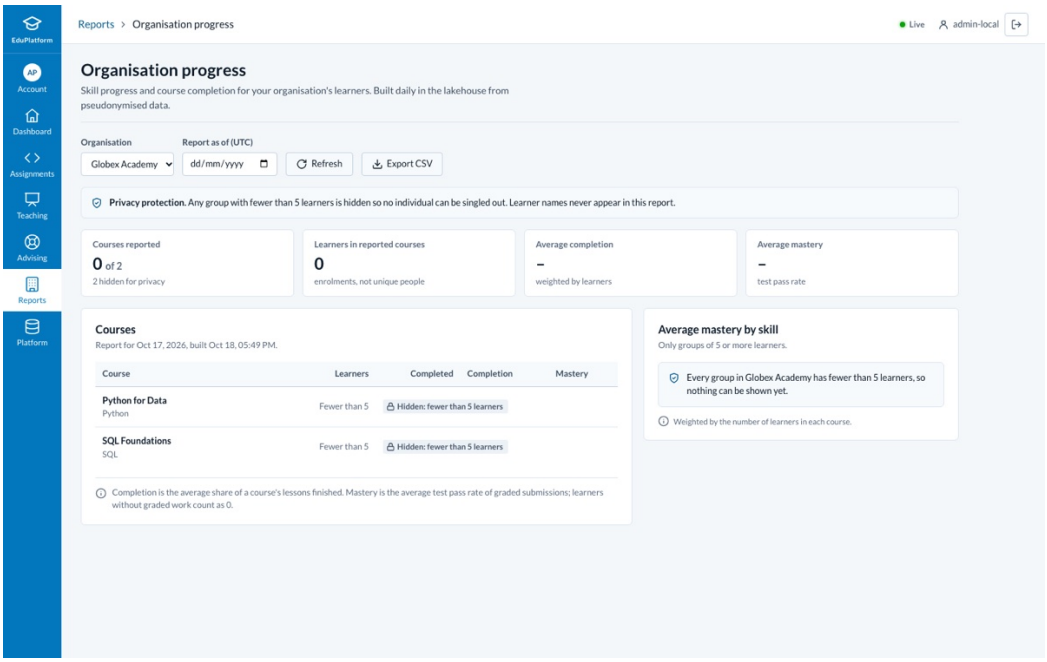
Hình 4.8. Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video "Joins" (390 giây), kèm các thời điểm cần xem lại



Hình 4.9. Learners to support (BR5): hàng đợi rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15

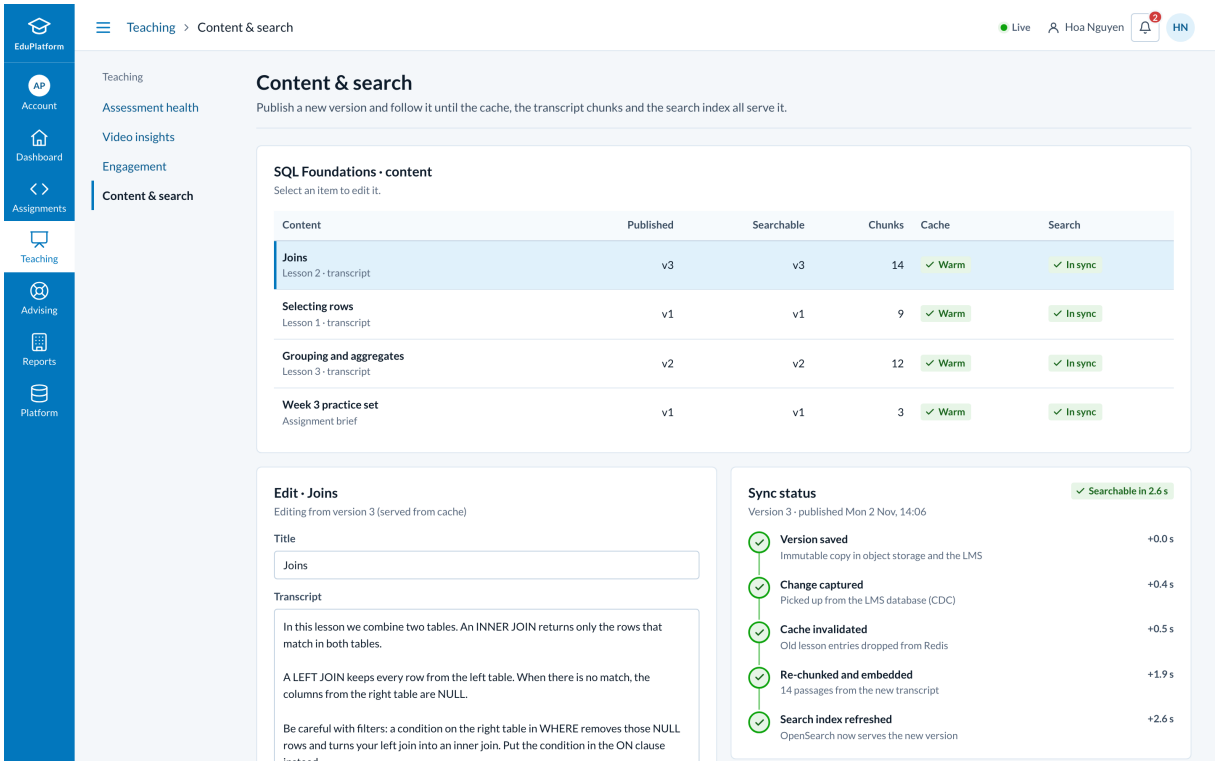


(a) Acme Learning: mọi nhóm đủ 5 người nên được báo cáo

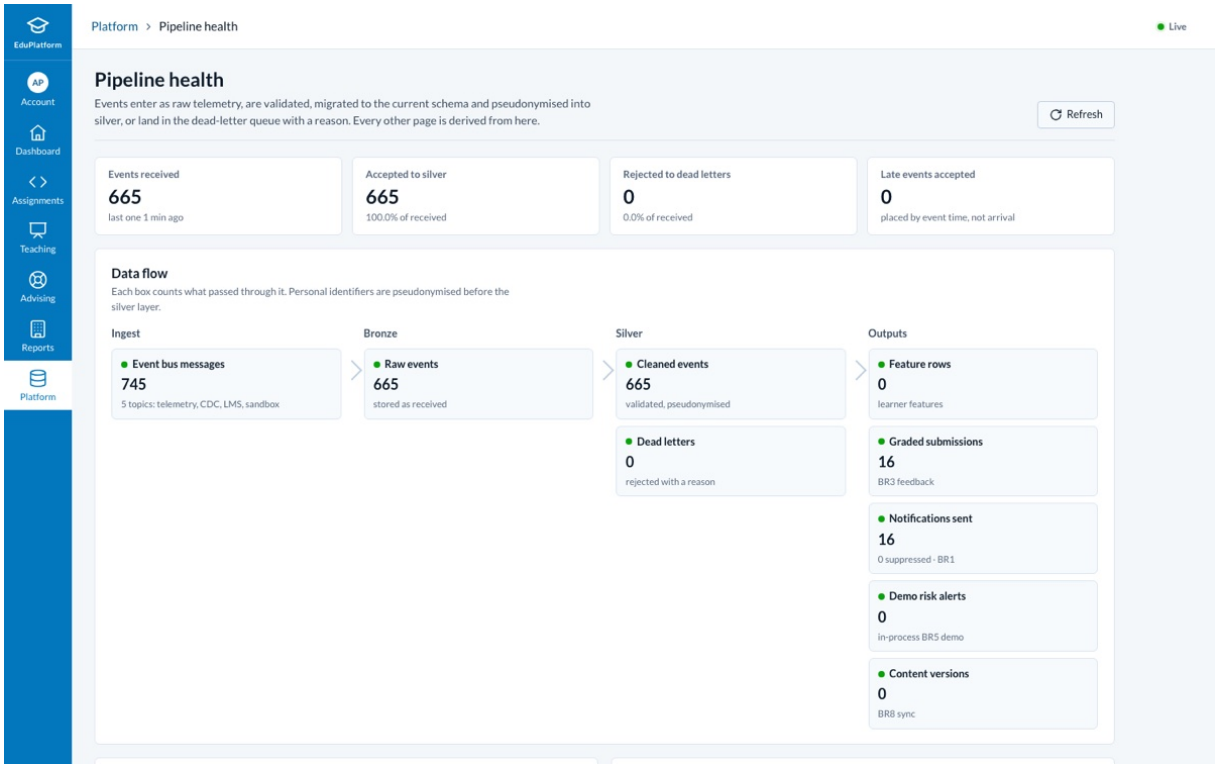


(b) Globex Academy: mọi nhóm dưới 5 người nên bị ẩn, không lộ tỷ lệ hay quy mô chính xác

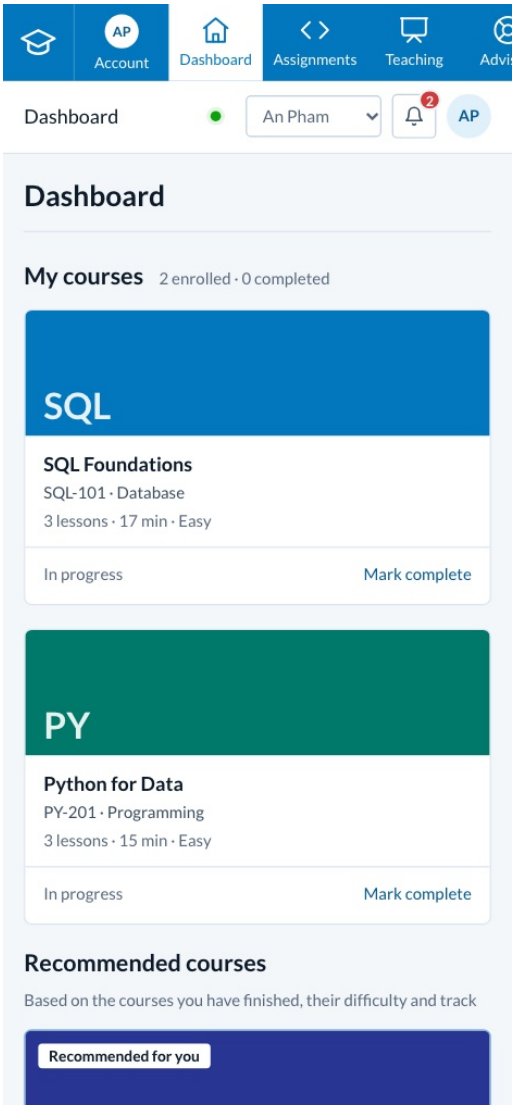
Hình 4.10. Organisation progress (BR6): k-anonymity với $k = 5$



Hình 4.11. Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được



Hình 4.12. Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu



Hình 4.13. Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang

4.2.6 Kiểm thử giao diện

Giao diện được kiểm chứng theo ba cách:

- **Biên dịch nghiêm ngặt**: TypeScript `strict`; bản build production thành công.
- **Kiểm tra thủ công trên trình duyệt** với dữ liệu thật ở chế độ sáng/tối, desktop và điện thoại: một bài nộp đi qua sandbox sinh thông báo mới; các trang BR6–BR8 đọc mart thật từ Trino.
- **Kiểm thử chấp nhận bằng Playwright** trên stack thật, gồm 12 test trong năm tệp spec: 2 test BR1–BR3, 3 test BR4/BR5, 4 test BR6–BR8, 2 test cho hai chế độ classroom và replay của BR4/BR5 và 1 test phiên đăng nhập nhân sự. Chín test của ba bộ đầu đã đạt ở các lần chạy trước khi gộp nhánh BR4/BR5; kết quả JSON và ảnh chụp được lưu kèm. Trên mã đã gộp, các test này đăng nhập bằng tài khoản nhân sự và cả bộ 12 test chưa được

chạy lại.

Chương 5

Kiểm thử và đánh giá

Chương này đánh giá tính đúng đắn và hiệu năng của hệ thống. Mọi số liệu đo trên dữ liệu tổng hợp, trên máy cá nhân với stack Docker Compose chạy đồng thời (một số dịch vụ không cần cho từng phép đo được tạm dừng do giới hạn bộ nhớ, xem Mục 5.3); đây là quan sát cục bộ, không phải cam kết năng lực. Các tệp CSV gốc lưu mọi mẫu đo (Phụ lục A).

5.1 Chiến lược kiểm thử

Chiến lược kiểm thử chia thành nhiều tầng, từ domain thuần đến dịch vụ thật và trình duyệt; Bảng 5.1 tóm tắt số lượng và phạm vi từng tầng. Số lượng được đếm trên mã đã gộp nhánh BR4/BR5; trên mã này, đợt cập nhật báo cáo chỉ chạy lại tầng đơn vị, nên con số của các tầng còn lại là số test hiện có chứ không phải kết quả chạy.

Mã Python được kiểm tra bằng `ruff` và `mypy`; kho dữ liệu chạy đủ 10 cổng chất lượng sau mỗi lần build.

Kiểm thử đơn vị tập trung vào policy và use case; kiểm thử tích hợp dùng PostgreSQL, Kafka và Redis thật; kiểm thử trình duyệt chạy trên stack đầy đủ. Cách phân tầng này giữ phản hồi nhanh ở tầng dưới và vẫn kiểm chứng được luồng đầu cuối.

Bảng 5.1. Các tầng kiểm thử

Tầng	Số lượng	Phạm vi
Đơn vị	379	Policy và use case của domain, phân quyền API với adapter giả, phiên đăng nhập nhân sự, phân tích bản ghi Debezium, S3 (moto), Redis (fakeredis), OpenSearch adapter, lớp phủ, cache; test kiến trúc kiểm tra ranh giới các lớp
Spark	11	SQL của bộ đọc mart trên bảng Delta dựng từ DDL; job reindex trên Delta với S3 giả lập; tính tương đương của lớp phủ streaming trên Kafka và Redis thật; BR4: sửa điểm trên Delta giữ ảnh chụp cũ và khớp phép tính đối chiếu; BR5: checkpoint cửa sổ khử trùng lặp sau khi khởi động lại
Tích hợp BR4/BR5	21	PostgreSQL và API: SQL khớp Python, lần làm đầu/cuối, kết quả thiếu hoặc không hợp lệ, lỗi hạ tầng và lỗi mã, đổi phiên bản, LSN cũ, cách ly sự kiện trễ, phân quyền, vòng đời cảnh báo, lease của job tổng hợp, phiên bản chính sách, ghi chú can thiệp, retention
Tích hợp BR1/BR3	1	HTTP end-to-end: enroll, nộp bài, chờ `graded`, kiểm tra thông báo từ `grading.submission_graded.v1`
Dịch vụ thật	–	OpenSearch 2.17 (analyzer, HNSW, lọc kNN, chỉ phiên bản hiện hành); danh mục PostgreSQL (5 người xuất bản đồng thời, rollback); tính tương đương lớp phủ kể cả gửi lại và phát lại toàn bộ
End-to-end	1 + 6	Luồng PostgreSQL → WAL → Debezium → Kafka → Spark → API với 6 kiểm tra chấp nhận, gồm sửa điểm, phát lại Kafka, UPDATE/DELETE hoạt động và khởi động lại checkpoint
Trình duyệt	12	Playwright: 2 test BR1–BR3, 3 test BR4/BR5, 4 test BR6–BR8, 2 test cho chế độ classroom và replay của BR4/BR5 và 1 test phiên đăng nhập nhân sự; drill-down của giảng viên và cập nhật khi sửa điểm, xác nhận cảnh báo và hồi phục rủi ro, lỗi đăng nhập, k-anonymity, heatmap theo phiên bản, xuất bản đến khi tìm kiếm được, lớp phủ trực tiếp

5.2 Kiểm chứng nghiệp vụ

Mỗi BR có ít nhất một kịch bản kiểm tra quy tắc nghiệp vụ và được quan sát đến giao diện.

Bảng 5.2. Các kịch bản kiểm chứng nghiệp vụ và kết quả

BR	Kịch bản	Kết quả quan sát
1	Đăng ký `l-002` vào `c-ml-301`, gửi deadline reminder, phát lại cùng sự kiện	PostgreSQL có enrollment; Redis inbox có đúng một thông báo và `count=1`; lần phát lại trả `duplicate`, không tăng bộ đếm; trong đợt kiểm chứng, email và webhook đều được gửi thành công
2	5 sự kiện telemetry live, thêm 2 sự kiện, chạy lại; build kho rồi materialise	Phiên có 5 rồi 7 sự kiện, cùng `started_at`; replay không đếm lại; Redis có 40 khoá longterm; API trả cả hai nhóm feature
3	Nộp bài qua BFF; phát lại queue/raw result; bật projector	`queued` → `running` → `graded`, pass rate 0,8; `grading.*` có dòng; inbox có thông báo; replay không tạo grade thứ hai; projector 9/9 → 11/11 rồi trả baseline
4	24 học viên, 3 test bắt buộc; 2 người đạt hết, 22 người trượt cùng một test	Câu hỏi đạt $2/24 = 8{,}33\%$; hai test đạt 100%, test lỗi đạt 8,33% và thấp hơn trung vị 91,67 điểm %: gắn cờ `SUSPICIOUS_TEST_CASE`. Sửa một kết quả trong PostgreSQL làm số hiển thị đổi từ 8% sang 13% mà không tải lại trang.
4	Kho dữ liệu: test hỏng `t5_pairs` và test khó nhưng hợp lệ `t3_ties`	Chỉ `t5_pairs` (0/68 lượt đạt) bị gắn cờ; `t3_ties` (0,55) không bị gắn cờ
5	Học viên có 4 hoạt động tuần trước, không hoạt động tuần này, một bài đến hạn chưa nộp	100 điểm, mức CRITICAL, sinh cảnh báo. Bốn hoạt động thật ghi qua API làm điểm giảm còn 50 và cảnh báo tự giải quyết. Phát lại Kafka không sinh cảnh báo trùng.
5	Kho dữ liệu: rủi ro theo ngày của một học viên	Một cảnh báo khi vượt ngưỡng; học viên bỏ học 6 ngày sau, nhãn kết quả `dropped_within_30d = true`
6	Globex có các nhóm 2 người; Acme có nhóm lớn	Globex chỉ công bố quy mô; Acme được báo cáo; truy cập ngoài phạm vi trả về 403
7	Video `v-sql-joins` được quay lại	Hai heatmap riêng: v1 (420 giây) và v2 (390 giây)
8	Xuất bản transcript có chuỗi đánh dấu duy nhất	Phiên bản mới tìm kiếm được, cache trả `miss` rồi được làm ấm; xoá qua CDC gỡ vector và mục tìm kiếm

BR	Kịch bản	Kết quả quan sát
–	Chạy kho dữ liệu: backfill, hai lần tăng dần, rồi chạy lại	Dấu vân tay các bảng giống hệt sau lần chạy lại (idempotent); một lần đồng bộ mobile trễ tự kéo cửa sổ gold lùi hai ngày

Ba kịch bản BR1–BR3 chạy trên stack thật với Kafka, Redis, PostgreSQL và Spark: integration test đạt và Playwright 2/2 đạt. Trên mã đã gộp nhánh BR4/BR5, cả 379 unit test đạt, không có test nào bị bỏ qua.

5.3 Môi trường đo

Toàn bộ phép đo chạy ngày 13/11/2026 trên một máy Apple M1 Pro (10 nhân CPU: 8 nhân hiệu năng và 2 nhân tiết kiệm điện), 32 GB RAM, macOS 26.6.2. Docker Desktop (Docker Engine 29.7.2) được cấp 10 CPU và 8 GB RAM (máy ảo báo 7,75 GiB), kèm 1 GiB swap; container Hive Metastore dùng image amd64 nên chạy giả lập trên ARM. Stack Docker Compose đầy đủ (`docker-compose.yml` và `docker-compose.br13.yml` với các profile `br45`, `br678`, `airflow`, `pipelines`) gồm 30 container chạy lâu dài: Kafka, Kafka UI, Schema Registry, Debezium, PostgreSQL, Redis, MinIO, Hive Metastore cùng cơ sở dữ liệu của nó, Trino, Flink (jobmanager, taskmanager), Spark (master, worker), OpenSearch, Airflow (webserver, scheduler, metadata database), API demo, các dịch vụ notification, grading (ingest, worker, normalizer), push-sink, Mailpit, content-sync, telemetry-live, lakehouse-streaming, br45-cdc và br45-stream.

Các phép đo chia thành hai đợt trong cùng ngày, trên cùng máy và cùng cấu hình Docker. Đợt đầu đo toàn bộ BR1–BR8 và phần so sánh Debezium với polling trên mã trước khi gộp nhánh BR4/BR5 (commit `4d70703`). Sau đó nhánh `Benchmark_BR45` được gộp vào `main` (commit `05bb5af`); lần gộp này thay đổi chính các đường mà hai benchmark BR4 và BR5 đo, nên riêng hai phép đo này được đo lại trên mã đã gộp: BR4 từ 21:17:33 đến 21:18:13 và BR5 từ 21:19:23 đến 21:20:42 ngày 13/11/2026 (giờ Việt Nam, UTC+7). Mọi số liệu của hai thí nghiệm quy mô BR4 và BR5 trong chương này lấy từ lần đo lại đó. Các phép đo còn lại (BR1–BR3, BR6–BR8, build kho, Debezium và polling) giữ kết quả của đợt đầu trên commit `4d70703` và chưa được đo lại trên mã đã gộp.

Với 8 GB, stack đầy đủ không chạy ổn định: trong 25 phút đầu của đợt đo đầu, khi thử các cấu hình 24–27 container, bộ nhớ khả dụng của máy ảo nhiều lần về gần 0 và Trino bị dừng đột ngột rồi tự khởi động lại 14 lần (dấu hiệu của OOM-kill). Vì vậy mỗi phép đo chạy trên một cấu hình rút gọn, và kết quả chỉ được giữ khi cấu hình đó đứng vững suốt cửa sổ đo. Thứ tự tạm dừng là: trước hết các job streaming mà phép đo không dùng (lakehouse-streaming, br45-cdc,

br45-stream, telemetry-live); nếu vẫn thiếu bộ nhớ thì đến các dịch vụ nhàn rỗi không nằm trên đường đo (cụm Flink chưa có job, Kafka UI), rồi đến Airflow (mọi DAG đang tạm dừng); riêng build kho còn phải dừng OpenSearch và content-sync. Bảng 5.3 ghi cấu hình của từng phép đo; hàng BR4/BR5 là cấu hình của lần đo lại trên commit `05bb5af`, các hàng còn lại thuộc đợt đầu trên commit `4d70703`.

Trạng thái mọi container được chụp lại ngay trước và ngay sau mỗi phép đo. Một lần đo chỉ được giữ khi tập container đang chạy, số lần khởi động lại và thời điểm khởi động của từng container không đổi trong suốt cửa sổ đo; các lần đo có container bị dừng giữa chừng đều bị loại (cột ghi chú của Bảng 5.3). Bảng 5.4 tóm tắt môi trường cho BR4–BR8.

Bảng 5.3. Container chạy trong từng phép đo (stack đầy đủ: 30 container)

Phép đo	Chạy	Tạm dừng trong lúc đo	Ghi chú
BR1–BR3; BR6–BR8 phục vụ mart; BR8 xuất bản; Debezium và polling	26	lakehouse-streaming, br45-cdc, br45-stream, telemetry-live	Chỉ tạm dừng các job streaming mà phép đo không dùng
BR4; BR5 (đo lại trên commit `05bb5af`)	23	lakehouse-streaming, telemetry-live, Flink (jobmanager, taskmanager), Kafka UI, Airflow (scheduler, webserver)	Cùng tập container như lần đo BR4/BR5 của đợt đầu (khi đó Airflow chạy cùng br45-stream chỉ để lại khoảng 560 MiB bộ nhớ khả dụng nên được tạm dừng). Ở lần đo lại, bộ nhớ khả dụng lúc bắt đầu đo là khoảng 740 MiB (BR4) và 590 MiB (BR5), swap 1 GiB đã đầy
BR7/BR8 lớp phủ	23	lakehouse-streaming, br45-cdc, br45-stream, Flink (jobmanager, taskmanager), Airflow (scheduler, webserver)	Hai lần đo khi Airflow còn chạy (27 và 24 container) bị loại vì Trino khởi động lại giữa chừng
Build kho dữ liệu	19	như lớp phủ, thêm Kafka UI, OpenSearch, content-sync, telemetry-live	Hai lần thử với 24 và 22 container bị loại vì tiến trình Spark bị dừng giữa chừng (dấu hiệu của OOM-kill)

Bảng 5.4. Môi trường đo hiệu năng

	BR4 / BR5 (13/11/2026, đo lại trên commit `05bb5af`)	**BR6–BR8 (13/11/2026, commit `4d70703`)**
Máy	Apple M1 Pro (10 nhân), 32 GB, macOS 26.6.2; Docker Desktop: 10 CPU, 8 GB (7,75 GiB)	Apple M1 Pro (10 nhân), 32 GB, macOS 26.6.2; Docker Desktop: 10 CPU, 8 GB (7,75 GiB)
Engine	Spark 3.5.5 và Delta 3.3.2 (image `de261/spark`), `local[2]`, driver 1 GiB, state store RocksDB, container giới hạn 2 CPU và 2.304 MiB; PostgreSQL 16.15; script đo, phép tổng hợp Python (BR4) và phép tính lô (BR5) chạy ngoài Docker bằng Python 3.13	Spark 3.5.5, Delta 3.3.2 (build kho: `local[*]`, 10 luồng); Trino 478 (heap 1,2 GB); OpenSearch 2.17.1 (heap 512 MB)
Lặp lại	3 lần mỗi kích thước	2–30 lần tuỳ thí nghiệm
Dữ liệu	Tổng hợp	Tổng hợp

Ở lần đo lại BR4/BR5, hai image `de261/demo` và `de261/spark` được build lại từ commit `05bb5af`, và các migration bổ sung của schema `analytics` được áp dụng trước khi đo. Job br45-stream chạy đúng cấu hình Compose mặc định của mã đã gộp: Spark 3.5.5 `local[2]` trong container giới hạn 2 CPU và 2.304 MiB, với thư mục checkpoint mặc định của Compose (`br45-v2`), được mã nguồn tự gắn thêm dấu vân tay cấu hình `ff4b0f4a118a`; tệp Compose override ngoài repo từng dùng ở đợt đầu để đổi thư mục checkpoint không còn cần. Vì đây là checkpoint mới, job đọc lại topic `br45.activity.v1` từ offset sớm nhất Kafka còn giữ (3394); phép đo chỉ bắt đầu sau khi job đã bắt kịp cuối topic và các job tổng hợp BR4 trên Spark do lần seed sinh ra đã hoàn tất.

5.4 Kết quả hiệu năng

5.4.1 BR1–BR3: độ trễ đầu cuối

Ba đường BR1–BR3 được đo ngày 13/11/2026 trên stack nói trên (26/30 container, Bảng 5.3), sau khi loại các mẫu khởi động nóng. Mỗi số đo là độ trễ của một chặng cụ thể sau khi stack đã dựng xong và các dịch vụ đã ấm, không tính thời gian khởi động stack. BR1 đo từ lúc gửi sự kiện LMS đến khi thông báo đọc được qua API; BR2 đo thời gian phản hồi API gợi ý có feature store trực tuyến; BR3 đo thời gian nhận 202 và từ lúc nộp đến khi trạng thái `graded`. Mỗi mẫu

BR1/BR3 lặp 20 lần, BR2 lặp 30 lần, và cả phép đo lặp năm lượt liên tiếp; bảng gộp cả năm lượt (n = 100 cho BR1/BR3, n = 150 cho BR2). Trước mỗi mẫu BR1, khoá dedup và hộp thư của người học được xoá: API trả toàn bộ hộp thư, nên một hộp thư tích luỹ sẽ làm phép đo chậm dần chứ không phản ánh độ trễ của chặng. Trạng thái chấm bài trong Redis cũng được xoá trước mỗi mẫu BR3, vì số thứ tự lần nộp được tính bằng cách quét toàn bộ bài nộp của người học.

Bảng 5.5. Độ trễ BR1–BR3 (ms)

Đường đo	n	Trung vị	p90	Nhỏ nhất	Lớn nhất
BR1: sự kiện → thông báo đọc được qua API	100	12,4	14,5	8,3	60,9
BR2: API gợi ý có feature store	150	2,2	4,2	1,4	12,2
BR3: nộp bài → 202	100	14,2	17,4	12,2	25,5
BR3: nộp bài → `graded`	100	71,0	77,3	65,4	90,2

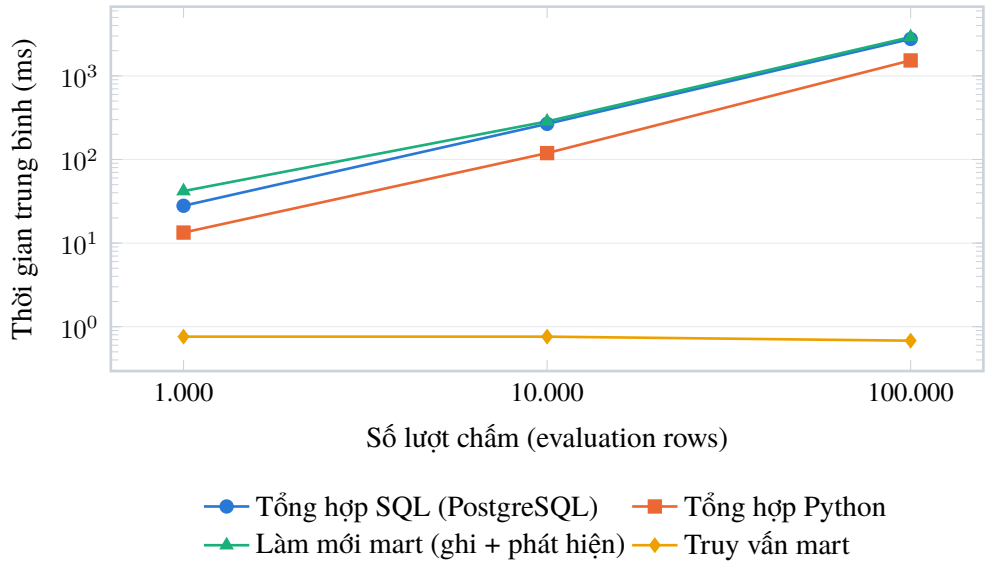
BR1 đạt trung vị khoảng 12 ms, phù hợp với yêu cầu thông báo gần thời gian thực. BR2 có trung vị khoảng 2,2 ms và p90 4,2 ms; 148/150 request nằm dưới ngưỡng 10 ms của yêu cầu tra cứu đặc trưng, hai request còn lại mất 10,5 và 12,2 ms. BR3 nhận 202 sau khoảng 14 ms và hoàn tất chấm sau khoảng 0,07 giây; sau khi trừ 14 ms nhận 202, phần còn lại chủ yếu là chạy sandbox và các chặng Kafka. Năm lượt đo cho kết quả ổn định, không trôi dần: trung vị BR1 12,1–13,6 ms, BR2 1,8–4,1 ms, BR3 13,7–14,4 ms và 70–72 ms; p90 chỉ cao hơn trung vị 2–6 ms. Hai giá trị lớn nhất của BR1 (54,8 và 60,9 ms) là các mẫu đơn lẻ.

5.4.2 BR4: tổng hợp SQL so với Python

Hai cách dùng cùng tập dữ kiện hợp lệ và cùng ngữ nghĩa chỉ số; kết quả khớp ở mọi kích thước và lần lặp. Hình 5.1 cho thấy thời gian tăng gần tuyến tính theo số lượt chấm. Python nhanh hơn khoảng hai lần (1,8–2,2 lần theo trung bình) nhờ dùng lại dữ kiện đã nối cho cả hai phép tổng hợp, còn SQL lặp lại một số phép nối JSONB. Ở 100.000 lượt chấm, một trong ba lần lặp của Python chậm hẳn (1,79 giây so với 1,38 và 1,42 giây), nên trung bình 1,53 giây cao hơn trung vị 1,42 giây; tính theo trung vị thì tỉ lệ ở kích thước này là 2,0 lần. Phép tổng hợp Python chỉ là đối chứng: trên đường sản phẩm, nhóm vẫn tổng hợp bằng SQL và công bố mart trong một giao dịch PostgreSQL vì ngữ nghĩa rõ ràng; truy vấn mart đã tổng hợp mất khoảng 0,7–0,8 ms.

Phép đo này chạy lại trên mã sau khi gộp (commit `05bb5af`, 23/30 container, Bảng 5.3) và chỉ đo đường PostgreSQL: "tổng hợp SQL" là hai truy vấn SQL của PostgreSQL trên `analytics.clean_rows` (JSONB), còn "làm mới mart" là hàm `refresh` của PostgreSQL, nay kết thúc

bằng bước công bố có khoá advisory và cổng chất lượng trên từng dòng. Trên mã đã gộp, engine tổng hợp mặc định của BR4 là Spark trên Delta (`EDU_BR45_ASSESSMENT_ENGINE=spark`), còn đường PostgreSQL là engine đối chứng; benchmark này không đo engine Spark.



Hình 5.1. BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log)

5.4.3 BR5: độ trễ đầu cuối và thông lượng cửa sổ

Lần chạy chấp nhận trên đường CDC thật mất khoảng **10.164 ms** từ khi ghi dữ liệu nguồn đến khi lưu kết quả rủi ro và cảnh báo. Khởi động lại Spark và hồi phục mất khoảng **20.142 ms**, không sinh cảnh báo trùng sau replay. Hai số này lấy từ lần chạy kịch bản chấp nhận trước đó (không đo lại). Thí nghiệm quy mô (Bảng 5.6) đo riêng chặng Kafka → Spark → cửa sổ trong PostgreSQL với khoảng 1% bản ghi trùng; mọi kết quả khớp với phép tổng hợp lô độc lập bằng Python. Thí nghiệm này được đo lại trên mã sau khi gộp (commit `05bb5af`, 23/30 container, Bảng 5.3) với job br45-stream của mã đó: Spark 3.5.5 dùng state store RocksDB, và cùng driver còn chạy thêm truy vấn landing Delta của BR4.

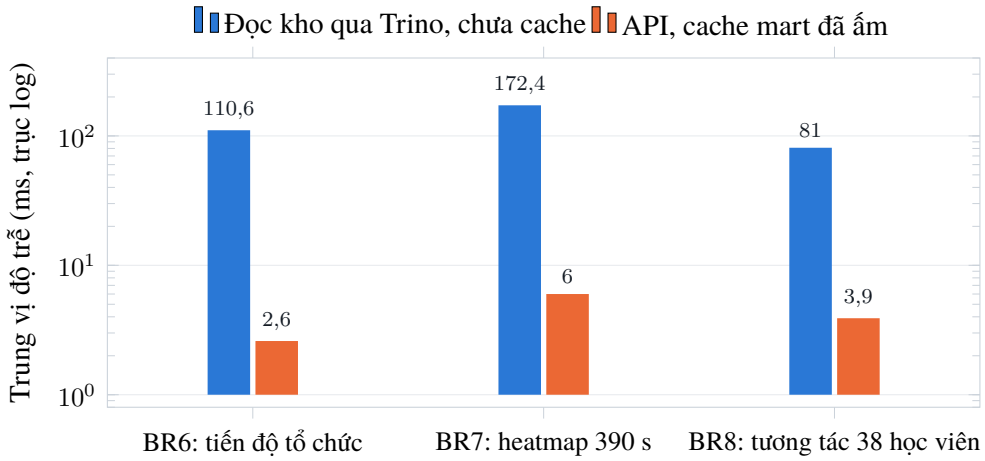
Bảng 5.6. BR5: chặng Kafka → Spark → cửa sổ đã lưu (trung bình 3 lần)

Sự kiện duy nhất	Kafka đến cửa sổ đã lưu	Thông lượng quan sát	Tính lô bằng Python
1.000	6.161 ms	170 sự kiện/s	1,8 ms
10.000	4.843 ms	2.088 sự kiện/s	18,4 ms
100.000	14.707 ms	6.800 sự kiện/s	150,2 ms

Ở 1.000 và 10.000 sự kiện, từng lần đo mất 4,1–8,2 giây và không tăng theo khối lượng: job chạy micro-batch với trigger 5 giây, nên độ trễ chủ yếu là thời gian chờ trigger. Tính lô rẻ hơn nhiều

cho ảnh chụp tĩnh; streaming đổi lấy tiếp nhận liên tục, xử lý sửa đổi, replay, checkpoint và cập nhật tự động, trong khi lô theo lịch phải chờ lượt chạy mới thấy thay đổi. Nhóm không đưa ra số liệu dương tính giả hay độ hiệu chỉnh điểm rủi ro vì không có nhãn bỏ học thật.

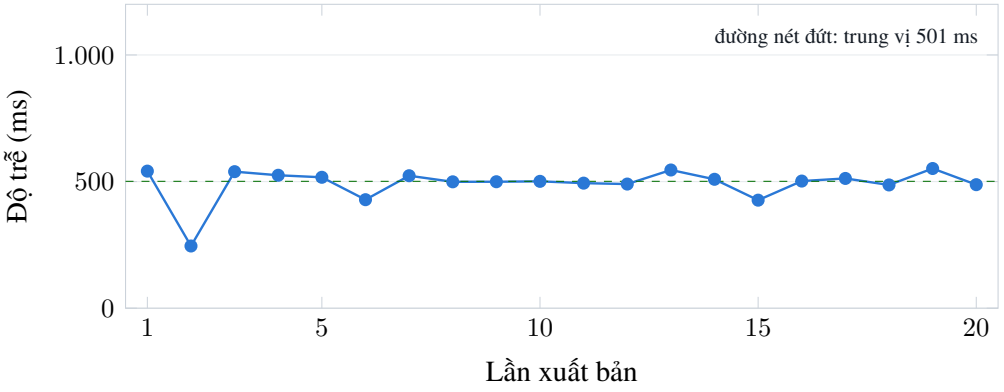
5.4.4 BR6–BR8: phục vụ mart



Hình 5.2. Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp)

Bách phân vị 90 khi đọc trực tiếp qua Trino là 220, 218 và 111 ms cho BR6, BR7, BR8; khi có cache là 3,4, 7,4 và 5,5 ms. Vì mart chỉ đổi mỗi ngày còn dashboard thăm dò 5 giây/lần, bộ đọc dùng cache TTL 60 giây (`CachedMartReader`). Cache cũng khắc phục lỗi Trino hết bộ nhớ khi dashboard thăm dò liên tục.

5.4.5 BR8: từ xuất bản đến khi tìm kiếm được



Hình 5.3. BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp

API nhận yêu cầu xuất bản trong 31,6 ms (trung vị). Toàn bộ đường đi PostgreSQL WAL → Debezium → Kafka → content-sync → tìm kiếm được có trung vị **501 ms**, p90 541 ms và lớn nhất 552 ms. Theo log của content-sync trong lúc đo, bước đồng bộ chỉ mất 40–85 ms; phần còn lại là thời gian bắt thay đổi và chu kỳ thăm dò của consumer.

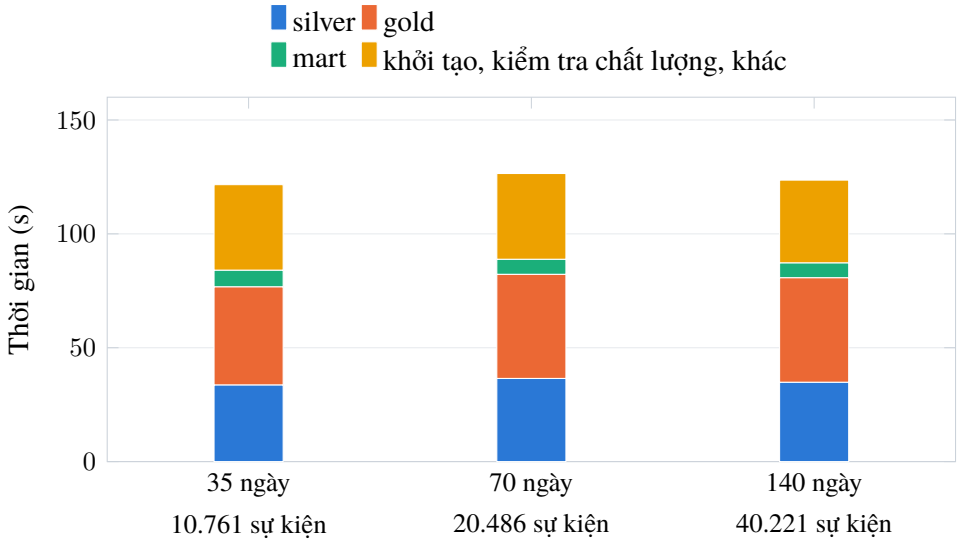
5.4.6 BR7/BR8: lớp phủ thời gian thực

Bảng 5.7. Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước)

Phiên mỗi đợt	Sự kiện thô (trung bình)	Trung bình (nhỏ nhất – lớn nhất)	Chính xác
25	234	5,71 s (4,76 – 6,93)	có
100	922	5,95 s (5,60 – 6,58)	có
400	3.618	6,13 s (5,36 – 7,16)	có

Độ trễ gần như không đổi khi tải tăng 15 lần (trung bình tăng từ 5,7 lên 6,1 s) vì chi phối bởi chi phí cố định của các micro-batch nối tiếp (trigger 1 s; theo log của telemetry-live trong lúc đo, mỗi micro-batch kéo dài khoảng 0,8–6,4 s), không phải khối lượng. "Chính xác" nghĩa là lớp phủ chứa đúng số tương tác đúng hạn của đợt cho video nhiều tương tác nhất.

5.4.7 Thời gian build kho dữ liệu



Hình 5.4. Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần)

Tổng thời gian build là 121,6; 126,5 và 123,6 giây cho 35, 70 và 140 ngày dữ liệu; mọi model đều thành công. Thời gian gần như phẳng khi khối lượng tăng 4 lần vì chi phí cố định mỗi model (khởi tạo job Spark, commit Delta, kiểm tra chất lượng) chiếm phần lớn; mỗi mart BR6–BR8 chỉ mất 1,3–2,3 giây. Hai lần chạy cùng kích thước lệch nhau nhiều nhất 6,5 giây (123,3 và 129,8 giây ở 70 ngày). Mỗi lần build chạy trong một tiến trình Spark riêng, trên cấu hình 19 container của Bảng 5.3.

5.5 Debezium và polling: negative-control benchmark

Phần này không tìm giải pháp nhanh hơn mà kiểm tra điều gì xảy ra khi bỏ Debezium, thay bằng polling bảng nguồn theo `updated_at`. Đây là negative-control: polling hợp lý ở quy mô nhỏ nhưng cần chỉ ra điểm vỡ khi dữ liệu lớn.

5.5.1 Thiết lập

Bảng nguồn `bench.cdc_source` có 100.000 dòng, khoá chính, cột `updated_at` và index trên cột này (tạo bằng một lệnh `CREATE INDEX` sau khi nạp bảng, Phụ lục A). Debezium đọc WAL qua logical replication; polling chạy mỗi 0,5 giây với watermark `updated_at`. Sau snapshot, thực hiện một đợt gồm 100 UPDATE và 10 DELETE cứng rồi đo event nhận được, latency, query và số tuple PostgreSQL trả về. Latency của polling tính từ `updated_at` của dòng đến lúc poller đọc được; latency của Debezium tính từ mốc `ts_ms` mà connector gắn vào event đến lúc consumer nhận được. Phép đo chạy ngày 13/11/2026 trên cấu hình 26 container của Bảng 5.3.

5.5.2 Kết quả

Bảng 5.8. Debezium và polling trên cùng 100.000 dòng

Chỉ số	Debezium	Polling
Event nhận được	110	100
UPDATE	100	100
DELETE cứng	**10**	**0**
Latency p50	485,9 ms	14,4 ms
Latency p95	487,2 ms	16,1 ms
Query bảng nguồn (SELECT/scan)	0	16
Query p50	–	18,6 ms
Tuple trả về do query bảng	0	4.061

Polling có index nên không full scan; đây là so sánh công bằng. Tuy vậy polling vẫn thiếu 10 DELETE vì hard delete không còn dòng để `SELECT`; Debezium nhận đủ 100 UPDATE và 10 DELETE. Về độ trễ, lần đo này không cho thấy Debezium nhanh hơn: polling đọc được 100 UPDATE sau trung vị 14,4 ms, còn event của Debezium đến consumer sau trung vị 485,9 ms.

Lưu ý: snapshot ban đầu của Debezium đọc 100.000 dòng nhưng nằm ngoài cửa sổ đo. "0" trong bảng là 0 query bảng trong cửa sổ đo, không phải 0 SQL tuyệt đối; Debezium vẫn dùng replication protocol và metadata. Tương tự, `tup_returned` của polling là số tuple index/heap PostgreSQL trả về, không phải số event.

5.5.3 Ý nghĩa với Big Data

Speed. Phép đo này không chứng minh được Debezium nhanh hơn polling. Trong lần đo, polling có độ trễ thấp hơn nhiều (p50 14,4 ms so với 485,9 ms của Debezium; p95 16,1 ms so với 487,2 ms). Cả hai giá trị đều nằm trong một chu kỳ 0,5 giây: polling thăm dò mỗi 0,5 giây và đợt UPDATE rơi ngay trước một lần thăm dò; độ trễ gần 0,5 giây của Debezium phù hợp với `poll.interval.ms` mặc định 500 ms của connector, vốn không được chỉnh trong cấu hình đo. Vì chỉ có một đợt thay đổi, mỗi con số là một mẫu về vị trí của đợt thay đổi trong chu kỳ, không phải phân phối độ trễ; muốn so sánh tốc độ cần lặp nhiều đợt ở các pha khác nhau.

Correctness. Polling chỉ thấy trạng thái hiện tại; hard DELETE biến mất khỏi bảng nên không có gì để poll. Nó cũng có thể bỏ qua UPDATE trung gian nếu một dòng bị sửa nhiều lần giữa

hai lần poll. Debezium giữ chuỗi thay đổi theo LSN, gồm cả DELETE và thứ tự giao dịch.

Stateful và retry. Debezium lưu offset/LSN và schema history trong Kafka Connect, nên restart có thể tiếp tục từ vị trí đã đọc. Polling không có trạng thái, phải tự lưu watermark; nếu poller chết giữa chừng hoặc commit watermark trước khi xử lý, có thể mất hoặc lặp event. Hard DELETE vẫn không thể phục hồi bằng watermark.

Scale dọc và ngang. Polling tăng tải theo số poller và số dòng/index phải quét; thêm poller là thêm query lên cùng bảng nguồn. Debezium đọc WAL với tải nguồn thấp và mở rộng theo partition/topic, consumer; nguồn không phải scan lại bảng khi dữ liệu tăng.

Ở 100.000 dòng, polling còn dùng được; ở 1–10 triệu dòng, chi phí index scan, số tuple trả về và số poller tăng tuyến tính trong khi Debezium vẫn đọc log. Với BR4–BR5 cần bắt sửa điểm, xoá và giữ thứ tự LSN, polling không đáp ứng đúng nghĩa dữ liệu; Debezium phù hợp hơn vì tính đúng đắn, không phải vì tốc độ: phép đo này không cho thấy Debezium nhanh hơn.

5.5.4 Giới hạn phép đo

Đây là smoke test 100.000 dòng trên một máy, chưa phải 1–10 triệu dòng. Chưa chạy failure injection cho connector. Độ trễ của cả hai phương án chỉ đo trên một đợt thay đổi nên phụ thuộc vào pha của chu kỳ thăm dò 0,5 giây và không dùng để xếp hạng tốc độ. Các số này đủ để chứng minh correctness/state/scale, không đủ tuyên bố SLA production.

Riêng BR1–BR3, phép đo chạy trên cùng một máy đang gánh phần lớn stack (26/30 container) nên chịu ảnh hưởng của tranh chấp CPU và bộ nhớ. Hai đặc điểm của bản demo ảnh hưởng trực tiếp đến cách đo và là hạn chế cần xử lý: API hộp thư trả toàn bộ danh sách, nên hộp thư càng dài mỗi lần đọc càng chậm (152 thông báo: khoảng 68 ms, so với 7,5 ms khi còn 10 thông báo), cần phân trang; và số thứ tự lần nộp được tính bằng cách quét toàn bộ bài nộp của người học, nên chi phí nhận 202 tăng theo lịch sử (398 bài nộp: khoảng 167 ms, so với 28 ms khi chưa có lịch sử). Vì vậy mỗi mẫu BR1/BR3 được đo trên trạng thái rỗng. Hai cặp số chẩn đoán trên lấy từ lần đo trước trên máy khác (không đo lại).

5.6 Thảo luận

- **Đáp ứng yêu cầu độ trễ.** Cảnh báo BR5 khoảng 10 giây (kịch bản chấp nhận, không đo lại) và tìm kiếm BR8 khoảng nửa giây đều đủ cho nghiệp vụ (cố vấn can thiệp, người học tìm nội dung mới). Lớp phủ khoảng 6 giây phù hợp dashboard trên mart theo ngày; muốn dưới một giây sẽ cần Flink (ADR 0014). Riêng ngưỡng 10 ms của BR2 đạt ở trung vị và p90, nhưng 2/150 request vượt ngưỡng.

- **Tách tính toán khỏi phục vụ.** Mart tổng hợp sẵn và cache giúp độ trễ phục vụ vài mili-giây dù truy vấn gốc qua Trino mất khoảng 80–170 mili-giây; tra cứu theo khoá vẫn ở Redis.
- **Đúng đắn quan trọng hơn tốc độ.** Ở BR4, nhóm giữ SQL dù Python nhanh hơn trên đường PostgreSQL và ghi rõ kết quả benchmark thay vì tuyên bố SQL thắng; hướng tối ưu (bảng có kiểu, dùng chung kết quả trung gian) đã được hiện thực trong engine mặc định Spark SQL trên Delta, nhưng engine này chưa có trong các phép đo của chương.
- **Lỗi phát hiện nhờ kiểm chứng.** Quá trình đo và kiểm thử đã phát hiện, sửa nhiều lỗi thật, ví dụ: bộ chia đoạn bỏ mất khoảng 2/3 transcript; đồng hồ thiết bị chạy nhanh làm rơi sự kiện đúng hạn của người khác; cache tài liệu bị đọc trước khi kiểm tra phạm vi; Trino hết bộ nhớ khi dashboard thăm dò liên tục.

Chương 6

Hạn chế, hướng phát triển và kết luận

Chương này nêu các hạn chế còn lại, hướng phát triển và kết luận, nhằm phân biệt phần đã hiện thực với phần còn là giới hạn hoặc thiết kế.

6.1 Hạn chế hiện tại

1. **Độ trễ streaming ở mức giây.** Cảnh báo BR5 sau khoảng 10 giây (không đo lại) và lớp phủ sau khoảng 6 giây, cao hơn mức dưới một giây của Flink. Đây là đánh đổi có chủ đích (ADR 0009, 0014).
2. **Embedding mang tính từ vựng.** Bộ nhúng băm không hiểu ngữ nghĩa ("join" không khớp "merge"); cổng `Embedder` và đường reindex cho phép thay bằng mô hình thật.
3. **Lớp phủ không mang phiên bản nội dung.** Khi video được quay lại, lớp phủ trộn hai phiên bản nên chỉ hiển thị cho phiên bản mới nhất.
4. **Lịch sử BR5 trong kho dữ liệu vẫn là dữ liệu tổng hợp.** Runtime BR5 nay ghi mỗi kết quả chấm điểm mới, mỗi cảnh báo mới và mỗi lần chuyển trạng thái cảnh báo vào outbox `analytics.domain_outbox` rồi phát lên `risk.learner_score.v1`, `risk.dropout_alert.v1` và `risk.alert_transition.v1`; landing của lakehouse nhận các topic `risk.*` và kho có model silver cho cả ba. Tuy vậy các lần build kho trong báo cáo vẫn chạy trên dữ liệu sinh tổng hợp, chưa nạp sự kiện từ runtime thật, và bảng `silver.dropout_alert_transitions` chưa được model gold nào dùng.
5. **Hiệu năng BR4.** Mỗi lần sửa vẫn tính lại toàn bộ bản tổng hợp 30 ngày của khoá học bị ảnh hưởng, không tính tăng dần. Trên engine đối chứng PostgreSQL, vốn dùng phép nối JSONB, với 100.000 lượt chấm, một lần sửa tốn khoảng 2,9 giây; engine mặc định Spark SQL trên Delta chạy bất đồng bộ và chưa được đo ở quy mô này trong báo cáo.
6. **Vận hành đơn nút, bảo mật cục bộ.** Trino, OpenSearch và Kafka chạy một nút; plugin

bảo mật OpenSearch tắt; nhân sự đăng nhập bằng tài khoản cấu hình tĩnh (mật khẩu băm Argon2id, phiên giữ trong bộ nhớ của một tiến trình API) và script dùng token tĩnh, thay cho hệ thống định danh của trường; chưa có giám sát độ trễ consumer, thời gian checkpoint hay SLA độ tươi.

7. **Dữ liệu tổng hợp, không nhãn thật.** Thiếu nhãn bỏ học nên chưa đánh giá được độ chính xác cảnh báo; các số đo hiệu năng chỉ là quan sát trên một máy.
8. **Lệch dữ liệu mẫu.** Seed lakehouse khai báo khoảng 18 khoá học và video không có trong LMS mẫu, nên job reindex báo lỗi.

6.2 Hướng phát triển

- Đưa stack đầy đủ thành mặc định khi triển khai; bổ sung retry và circuit breaker cho gửi thông báo.
- Chuyển job cần độ trễ dưới một giây sang Flink khi nền tảng lên Flink 2.x và connector Delta Kernel ổn định.
- Thay bộ nhúng băm bằng sentence-embedding và chạy một lần reindex.
- Chuyển bảng snapshot (`fct_enrollment`, `fct_lesson_progress`) sang MERGE dựa trên Change Data Feed khi khối lượng đủ lớn; với dashboard dưới giây ở quy mô lớn, bổ sung một engine OLAP chuyên dụng đọc qua Hive Metastore.
- Triển khai trên cụm nhiều nút với TLS và xác thực tập trung; bổ sung giám sát tập trung và quy trình xoay khoá bút danh.
- Thu thập nhãn kết quả thật để hiệu chỉnh và đánh giá điểm rủi ro BR5.

6.3 Kết luận

Báo cáo đã trình bày Project, nền tảng kỹ thuật dữ liệu biến bốn nguồn dữ liệu rời rạc của hệ thống giáo dục trực tuyến thành kho dữ liệu phục vụ tám yêu cầu nghiệp vụ.

Về kiến trúc, nền tảng bám sát kiến trúc tham chiếu: Kafka và Debezium cho tiếp nhận, Spark cho xử lý, Delta Lake cho lakehouse, Redis cho phục vụ độ trễ thấp; lớp gold là kho dữ liệu Kimball phục vụ qua Trino. Các điều chỉnh — đáng kể nhất là dùng Spark Structured Streaming thay Flink cho BR5 và lớp phủ — đều được ghi bằng ADR kèm lý do và hệ quả. Cách tổ chức DDD và kiến trúc lục giác cho phép cùng một logic nghiệp vụ chạy trên nhiều engine và kiểm thử độc lập.

Về hiện thực, BR4–BR8 chạy trên hạ tầng thật với các tính chất cốt lõi: xử lý theo thời gian sự

kiện, chịu được dữ liệu trễ, trùng lặp và sai thứ tự, chạy lại idempotent, khử định danh ở silver, k-anonymity cho báo cáo tổ chức và cổng chất lượng sau mỗi lớp. BR1–BR3 chạy end-to-end trên Kafka, Redis, PostgreSQL, Delta và Spark: sự kiện đi qua dịch vụ độc lập, kết quả đến giao diện qua SSE, mọi đường ghi đều idempotent; BR2 có đủ hai nhánh feature. Giao diện web mới đưa kết quả đến từng nhóm người dùng.

Về đánh giá, hệ thống có 379 test đơn vị, 1 test tích hợp end-to-end, 12 test trình duyệt và các kịch bản kiểm chứng nghiệp vụ quan sát đến giao diện. Số đo cho thấy độ trễ phù hợp từng yêu cầu: cảnh báo bỏ học khoảng 10 giây sau thay đổi nguồn (kịch bản chấp nhận, không đo lại), nội dung mới được lập chỉ mục sau khoảng nửa giây, mart phục vụ trong vài mili-giây khi có cache.

Bài học lớn nhất là giá trị của kiểm chứng bằng dữ liệu và số đo: nhiều lỗi quan trọng chỉ lộ ra khi chạy hệ thống thật. Việc ghi rõ phần đã hiện thực và phần còn là thiết kế giúp đánh giá đúng mức độ hoàn thiện và định hướng phát triển.

Tài liệu tham khảo

[1] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[2] Advanced Distributed Learning Initiative. Experience API (xAPI) Specification. `https://github.com/adlnet/xAPI-Spec`, 2017.

[3] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*, 2011.

[4] Debezium Community. Debezium: Stream changes from your database. `https://debezium.io/documentation/`, 2026. Truy cập 11/2026.

[5] Matei Zaharia, Reynold S. Xin, Patrick Wendell, et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11):56–65, 2016.

[6] Michael Armbrust, Tathagata Das, Joseph Torres, et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[7] Michael Armbrust, Tathagata Das, Liwen Sun, et al. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.

[8] Raghav Sethi, Martin Traverso, Dain Sundstrom, et al. Presto: SQL on Everything. In *IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813, 2019.

[9] Redis Ltd. Redis Documentation. `https://redis.io/docs/`, 2026. Truy cập 11/2026.

[10] OpenSearch Project. OpenSearch Documentation: k-NN and hybrid search. `https://opensearch.org/docs/latest/`, 2026. Truy cập 11/2026.

[11] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[12] Alistair Cockburn. Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/, 2005.

[13] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[14] Michael Nygard. Documenting Architecture Decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions, 2011.

[15] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4), 2015.

[16] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.

[17] Databricks. What is the medallion lakehouse architecture? https://docs.databricks.com/en/lakehouse/medallion.html, 2026. Truy cập 11/2026.

[18] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.

[19] Latanya Sweeney. k-Anonymity: A Model for Protecting Privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.

[20] Yu. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[21] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.

[22] Instructure. Canvas LMS Guides. https://community.canvaslms.com/t5/Canvas-Guides/ct-p/canvas_guides, 2026. Truy cập 11/2026.

[23] Moodle HQ. Moodle User Documentation. https://docs.moodle.org/, 2026. Truy cập 11/2026.

[24] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/, 2018.

Phụ lục A

Hướng dẫn chạy và tái lập kết quả

A.1 Yêu cầu

Docker (8 GB RAM cho VM đủ để chạy từng nhóm BR; chạy đồng thời cả bốn profile thì 8 GB không đủ, xem Mục 5.3), Python 3.11+, Node.js 20+. PostgreSQL mặc định ở cổng 5433 để tránh xung đột.

A.2 Khởi động

```
# BR1/BR3: full stack with Kafka, Redis, PostgreSQL and separate services
docker compose -f docker-compose.yml -f docker-compose.br13.yml up -d --build

# BR4/BR5: real CDC + Spark Structured Streaming
docker compose --profile br45 up -d --build demo br45-init br45-cdc br45-
    stream
docker compose exec -T br45-cdc python -m edu_platform.br45.cli seed

# BR6/BR7/BR8: warehouse, content sync, live overlays
docker compose --profile br678 up -d --build     # first run takes about 4-5
    minutes
docker compose exec demo edu telemetry simulate --sessions 50

# BR2: live session branch + materialise long-term features into Redis
docker compose --profile br678 run --rm --no-deps telemetry-live edu stream
    telemetry-live --once
docker compose run --rm spark-jobs edu batch features-online --run-date
    2026-11-11
```

```
# Warehouse with synthetic data
docker compose run --rm spark-jobs edu warehouse bootstrap
docker compose run --rm spark-jobs edu warehouse seed-bronze --end-date
    2026-10-17 --days 35
docker compose run --rm spark-jobs edu warehouse run --run-date 2026-10-17 --
    start-date 2026-08-01
docker compose exec trino trino --user analyst --catalog lakehouse
```

Mở ứng dụng tại `http://localhost:8000`. Overlay `br13` dựng các dịch vụ notification và grading trên Kafka, Redis, PostgreSQL.

Ba workspace Instructor, Advisor và Org admin yêu cầu đăng nhập bằng tài khoản nhân sự (`docs/STAFF_AUTH.md`). Compose mặc định không khai báo tài khoản nào (`EDU_STAFF_ACCOUNTS` rỗng) và chỉ khai báo một token Bearer của người học (`learner-demo`). Vì vậy cần tạo tài khoản và khai báo token cho script trong shell *trước khi* chạy các lệnh `docker compose ... up` ở trên; nếu `demo` đang chạy thì chạy lại lệnh `up` đó để tạo lại container với hai biến này.

```
# Staff accounts: each command asks for a password (12-128 characters)
# and prints one account with its Argon2id hash
umask 077; accounts_dir=$(mktemp -d)
.venv/bin/python -m edu_platform.br45.auth instructor instructor.demo \
  instructor-local "Demo Instructor" --course c-py-201 --course c-sql-101 \
  --course c-de-301 > "$accounts_dir/instructor.json"
.venv/bin/python -m edu_platform.br45.auth advisor advisor.demo \
  advisor-local "Demo Advisor" --course c-py-201 \
  > "$accounts_dir/advisor.json"
.venv/bin/python -m edu_platform.br45.auth admin admin.demo admin-local \
  "Demo Admin" --organisation org-acme --organisation org-globex \
  > "$accounts_dir/admin.json"
.venv/bin/python -m edu_platform.br45.auth admin acme-admin.demo \
  acme-admin-local "Acme Admin" --organisation org-acme \
  > "$accounts_dir/acme-admin.json"
export EDU_STAFF_ACCOUNTS="$(jq -cs 'add' "$accounts_dir"/*.json)"

# Bearer tokens for scripts (e2e check, BR6-BR8 benchmarks, Playwright API
# checks). Local demonstration values only: token -> identity and scope.
export EDU_BR45_AUTH='{
  "instructor-demo": {"id": "instructor-local", "role": "instructor",
    "courses": ["c-py-201", "c-sql-101", "c-de-301"]},
  "advisor-demo": {"id": "advisor-local", "role": "advisor",
    "courses": ["c-py-201"]},
  "admin-demo": {"id": "admin-local", "role": "admin",
    "organisations": ["org-acme", "org-globex"]},
  "acme-admin-demo": {"id": "acme-admin-local", "role": "admin",
```

```
    "organisations": ["org-acme"]},
  "learner-demo": {"id": "br45-demo-at-risk", "role": "learner",
    "courses": ["c-py-201"]}}'
```

Đăng nhập trên giao diện bằng tên người dùng vừa tạo (`instructor.demo`, `advisor.demo`, `admin.demo` cho cả hai tổ chức, `acme-admin.demo` chỉ cho Acme) và mật khẩu đã nhập; phiên có hạn tám giờ và mất khi API khởi động lại.

A.3 Kiểm thử

```
.venv/bin/ruff check src tests && .venv/bin/mypy
.venv/bin/python -m pytest tests/unit                          # 379 unit tests
EDU_BR13_BASE_URL=http://localhost:8000 .venv/bin/python -m pytest -m
    integration tests/integration/test_br13_services.py
# 11 Spark tests; require JDK, pyspark and delta-spark (the two BR4/BR5 ones
# are skipped unless EDU_BR45_RUN_SPARK_TESTS=1)
.venv/bin/python -m pytest -m spark tests/integration
# BR4/BR5 e2e: both tokens must be in EDU_BR45_AUTH of the demo service
BR45_VERIFY_INSTRUCTOR_TOKEN=instructor-demo \
  BR45_VERIFY_ADVISOR_TOKEN=advisor-demo \
  EDU_BR45_E2E=1 .venv/bin/python -m pytest tests/e2e/test_br45.py
cd apps/web && npm ci && npm run build && npx playwright install chromium
BR45_BASE_URL=http://localhost:8000 npx playwright test tests/br123.spec.ts
     # BR1--BR3
# All 12 tests: staff sign-in uses the password chosen above, the API
# checks use the Bearer tokens; the classroom, replay and staff-session
# tests are skipped unless their own BR45_BROWSER_* variables are set
BR45_BROWSER_STAFF_PASSWORD='<staff password>' \
  BR45_VERIFY_INSTRUCTOR_TOKEN=instructor-demo \
  BR45_VERIFY_ADVISOR_TOKEN=advisor-demo BR45_VERIFY_ADMIN_TOKEN=admin-demo \
  BR45_VERIFY_ACME_ADMIN_TOKEN=acme-admin-demo \
  BR45_BASE_URL=http://localhost:8000 npx playwright test
```

A.4 Đo hiệu năng

Các phép đo dùng stack như mục khởi động: hai tệp Compose với các profile `br45`, `br678`, `airflow` và `pipelines`, đúng như cấu hình mô tả ở Chương 2. Với Docker 8 GB, stack đầy đủ không chạy ổn định; trước mỗi phép đo cần tạm dừng (`docker compose stop`) các dịch vụ mà phép đo không dùng, theo đúng cấu hình ở Bảng 5.3, và kiểm tra không có container nào khởi động

lại trong lúc đo. Năm lượt BR1–BR3 được gộp thành `report/data/br123_latency.csv`; các tệp kết quả còn lại được sao chép vào `report/data/`. Trên mã đã gộp nhánh BR4/BR5, ba lệnh `edu benchmark serving`, `content` và `live` gọi API bằng hai token Bearer cố định trong mã (`admin-demo`, `instructor-demo`), nên dịch vụ `demo` phải chạy với `EDU_BR45_AUTH` như ở mục khởi động.

```
for i in 1 2 3 4 5; do                                  # BR1--BR3, 5 runs
  .venv/bin/python benchmarks/br123_latency.py \
    --samples 20 --br2-samples 30 \
    --out report/data/br123_latency_run$i.csv
done
.venv/bin/python -m edu_platform.br45.cli benchmark          # BR4
.venv/bin/python -m edu_platform.br45.stream_benchmark       # BR5
.venv/bin/edu benchmark serving --requests 30 \
  --out docs/evidence/br678/serving_latency.csv
.venv/bin/edu benchmark content \
  --out docs/evidence/br678/content_sync_latency.csv
.venv/bin/edu benchmark live --out docs/evidence/br678/live_overlay.csv
# Warehouse build: one process per (days, repeat) inside spark-jobs
docker compose run --rm -v "$PWD/docs/evidence/br678:/out" spark-jobs \
  edu benchmark marts --days 35 --repeat 1 --first-repeat 1 \
  --out /out/mart_build.csv     # repeat for 35/70/140 days, repeats 1-2
```

A.5 Đo Debezium và polling

Negative-control benchmark dùng bảng nguồn 100.000 dòng:

```
# Generate the facts CSV if missing (deterministic, seed 42)
.venv/bin/python benchmarks/cdc/generate_facts.py \
  --rows 1000000 --output benchmarks/cdc/data/org-facts-1m.csv

# Create the source table, then the updated_at index
# (setup_source does not create the index)
.venv/bin/python -m benchmarks.cdc.setup_source \
  benchmarks/cdc/data/org-facts-1m.csv --rows 100000
docker compose exec postgres psql -U lms -d lms -c \
  "CREATE INDEX cdc_source_updated_at_idx ON bench.cdc_source (updated_at)"

# Register the Debezium connector (once)
curl -sS -X DELETE http://localhost:8083/connectors/bench-cdc >/dev/null 2>&1
    || true
curl -sS -X POST -H 'Content-Type: application/json' \
```

```
  --data @benchmarks/cdc/connector.json http://localhost:8083/connectors

# Wait for the snapshot to reach 100,000 offsets
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --broker-list localhost:9092 --topic bench.bench.cdc_source

# Run 100 UPDATEs + 10 DELETEs
.venv/bin/python -m benchmarks.cdc.run_benchmark \
  --duration 8 --updates 100 --deletes 10 \
  --out benchmarks/cdc/results/cdc_100k_indexed.json
```

A.6 Biên dịch báo cáo

```
cd report
tectonic -X compile main.tex   # ặhoc: latexmk -xelatex main.tex
```

Trên Overleaf, chọn trình biên dịch XeLaTeX.

Phụ lục B

Tóm tắt API

Mọi endpoint có tiền tố `/api`; tài liệu OpenAPI ở `http://localhost:8000/docs` mô tả hợp đồng chi tiết. Endpoint phân tích yêu cầu xác thực: trình duyệt đăng nhập bằng tài khoản nhân sự qua `/auth/staff/{role}/session` rồi gửi cookie phiên kèm header `X-Staff-Role`, còn script gửi token Bearer đã khai báo trong `EDU_BR45_AUTH`. Thiếu hoặc sai thông tin xác thực trả 401, sai vai trò hoặc ngoài phạm vi trả 403, không có dữ liệu trả 404, truy vấn sai trả 422, kho chưa sẵn sàng trả 503.

Bảng B.1. Các endpoint chính

BR	Phương thức	Đường dẫn	Mục đích
1	GET	`/learners/{id}/notifications`	Thông báo của người học
2	GET	`/learners/{id}/recommendations`	Gợi ý và khoá đã ghi danh
3	POST	`/submissions`	Nộp bài (202, xếp hàng)
3	GET	`/submissions/{id}`	Kết quả và phản hồi
4	GET	`/instructor/courses/{course}/questions/statistics`	Thống kê câu hỏi
4	GET	`/instructor/questions/{question}/test-cases`	Thống kê test case
5	GET	`/advisor/courses/{course}/at-risk-students`	Hàng đợi rủi ro, tóm tắt mức
5	GET	`/advisor/students/{student}/risk-history`	Lịch sử điểm
5	GET	`/advisor/alerts`	Danh sách cảnh báo

BR	Phương thức	Đường dẫn	Mục đích
5	PATCH	`/advisor/alerts/{id}/acknowledge`, `/resolve`	Vòng đời cảnh báo
6	GET	`/admin/organisations/{org}/skill-progress`	Báo cáo tổ chức (k = 5)
7	GET	`/instructor/courses/{course}/videos`	Video và phiên bản
7	GET	`/instructor/videos/{id}/interaction-heatmap`	Heatmap kèm lớp phủ
8	GET	`/instructor/courses/{course}/engagement-trend`	Xu hướng tương tác
8	POST	`/instructor/content/{id}/versions`	Xuất bản phiên bản (202)
8	GET	`/instructor/courses/{course}/search`	Tìm kiếm lai hoặc từ khoá
–	POST, GET, DELETE	`/auth/staff/{role}/session`	Đăng nhập, đọc và huỷ phiên nhân sự
–	GET	`/events`	Server-Sent Events cho sự kiện trực tiếp